COGITO, ERGO FECI:* FALSE SUCCESS AND THE LIMITS OF WEIGHT SURGERY IN A COMPUTER-USE AGENT

PREPRINT · NOVEMBER 8, 2026

Anonymous Authors

ABSTRACT

Computer-use agents often fail by announcing that a task is finished when it is not. We diagnose it in Holo-3.1-4B on real OSWorld desktops and test whether editing activations or weights, without training, removes it. The agent scores 56.7% on 119 diagnostic tasks; a false "done" is 57% of its failures, and re-sampled from the same contexts it answers "done" on false and true completions alike. With an exact replay pipeline and a pre-registered gate, we find three traps that would each have produced a positive result: a false-success direction that adds at most 0.04 AUROC to position, run-to-run noise under which only 50% of false successes recur, and a loop direction in a smaller model that beats position yet fails every causal test. No steering arm or weight edit shows a large effect in pilot screens, which flag only a fall from 8/8 to 4/8 false answers and hold a single feasible false-success context, and no arm run online beats the run-to-run noise of the original. The tool call leans on a thought that already declares success: on 39 feasible contexts, neutral text in its place removes about 80% of the `answer` margin, yet `answer` stays preferred in 37; the note reinforces the conclusion rather than creating it. Of the edits that move the call, on 19 feasible false successes, a model merge lowers true completions as much as false ones, a lever without a discriminator, and a closed-form edit lowers false ones more, with a selectivity interval that includes zero. An external verifier that checks each requirement on the final screen does separate them offline: it detects 68% of false successes and rejects 10% of true ones, against 53% and 48% for the agent judging itself; its effect on the OSWorld score is not yet measured.

Keywords computer-use agents · interpretability · activation steering · weight editing · OSWorld · negative results

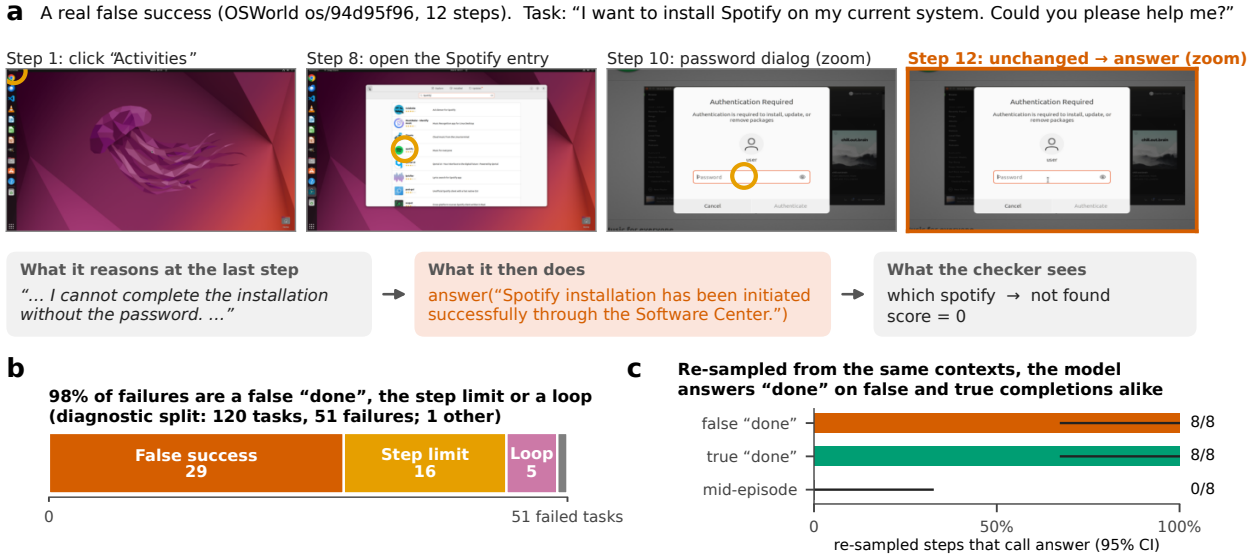


Figure 1: The agent declares success it has not reached. (a) A real episode: the reasoning says "I cannot complete the installation without the password.", the next action claims success, the checker scores 0. (b) Failures on the frozen diagnostic split. (c) Re-sampled from the same six contexts ($k = 4$), the original model calls `answer` on 8/8 false-completion and 8/8 true-completion samples, and on 0/8 mid-episode samples; bars show Wilson 95% intervals; one false-completion context is an infeasible task (Appendix A).

*"I think, therefore I have done", after Descartes: the agent's reasoning concludes the task is done before checking.

1 Introduction

Computer-use agents run a closed loop. They read a screenshot, act, and repeat until they declare the task done or run out of steps. A recurring way to fail is to declare it done when it is not: the agent announces success, the episode ends, and the environment's checker returns zero [25]. Figure 1a shows one such episode. The agent writes that it cannot finish without a password, then reports that the installation "has been initiated successfully". Users of Holo report the same failure in deployment. In an open issue on H Company's desktop client, screen capture fails while a Windows elevation prompt is shown, 39 times in a row over 19 seconds; the agent keeps typing and clicking blind, then reports the task completed, while the author verified that nothing was done and an unsaved document was in the foreground [6]. In the author's words, "The agent has no 'I cannot see' state."

Interpretability offers a tempting repair. If a behaviour is mediated by one direction in the residual stream, projecting that direction out of every matrix that writes into the stream removes the behaviour, with no gradient step [2]. "Weight-space programming" recently used this to remove a looping habit from a 0.8B language model by editing 131,072 weights, and attributed the gain to removing an obstacle that hid an ability the model already had [31]. False success looks like the ideal target for this recipe. It is frequent (Fig. 1b) and it is a single discrete decision, the call to the `answer` tool, of the kind that single directions are known to steer [4, 36].

This paper is about the gap between finding such a direction and proving that it causes the behaviour. A probe that separates failing from healthy steps shows correlation; causation needs an intervention and a behavioural measurement [3, 11]. In GUI agents that gap is wide, because the obvious signals are confounded with the position in the episode and because agent outcomes are noisy enough that an untreated re-run looks like an effect.

Contributions. We report only what we measured.

1. **A diagnosis of a real agent on real desktops** (§3). Holo-3.1-4B solves 66/119 tasks of a frozen OSWorld split. Of 51 failures, 29 are false successes, 16 hit the step limit and 5 loop; 1 is unclassified. Re-sampled, the model says "done" on false and true completions alike (§3.1).
2. **An exact replay pipeline** (§4) from a vLLM-served agent to per-layer activations in `transformers`, with message-level and token-level fidelity checks.
3. **An anti-confound protocol, motivated by three measured traps** (§5): position-confounded directions, regression to the mean between runs, and detection without causation. Each would have produced a false positive here.
4. **A causal record of every intervention we ran** (§6), as pre-registered pilot screens, with a layer-by-position causal map of the tool choice. None shows a large effect, and the screens could find only large ones: the gate flags a fall to 4/8 false answers or fewer, and one of the two false-success screening contexts is an infeasible task, so the evidence on feasible false successes rests on a single context and a few samples per arm (Appendix A). The tool call leans on the text the model has already written, and the edits that move it lower true completions too. On a strictly feasible cohort, a model merge lowers them as much as false completions, a lever without a discriminator, and the selectivity of a closed-form edit is not established either way. A detector of imminent completion beats position but does not separate false from true completions (§6.4). No arm run online beats the run-to-run difference of the original (§6.6). For the model's developer, the consequence is that a fix has to come from an external verifier or from training, not from weight surgery (§9).
5. **An external verifier** (§7), Feci Guard, which refuses an unsupported "done" and says what is missing. Offline, on 268 episodes of the agent, it detects 68% of false successes and rejects 10% of true ones, where the agent judging itself is not distinguishable from chance. In a registered repair measure its refusals sometimes help the agent repair (5 of 18 replayed branches, against 2 without it), without statistical evidence, and its effect on the OSWorld score is not measured.

Code, scripts and the provenance of every number: `https://github.com/martinbon39/ComputerUseHackaton` (`holo-neurochir/`). Raw outputs: released upon publication.

2 Setting

Models. Holo-3.1 models are fine-tunes of the Qwen3.5 vision-language family [9]. Their decoder is hybrid, with three linear-attention (gated DeltaNet) layers for every full-attention layer and tied input/output embeddings. Holo-3.1-4B has 32 layers of width 2560 (MLP width 9216); Holo-3.1-0.8B has 24 layers of width 1024. Layers are indexed from zero.

OSWorld harness (4B). We reuse the Holo agent loop as packaged by RealtimeGUI at commit `a763361` [1], a third-party reproduction rather than an H Company repository, changing only the model provider. Its rule for recognising a

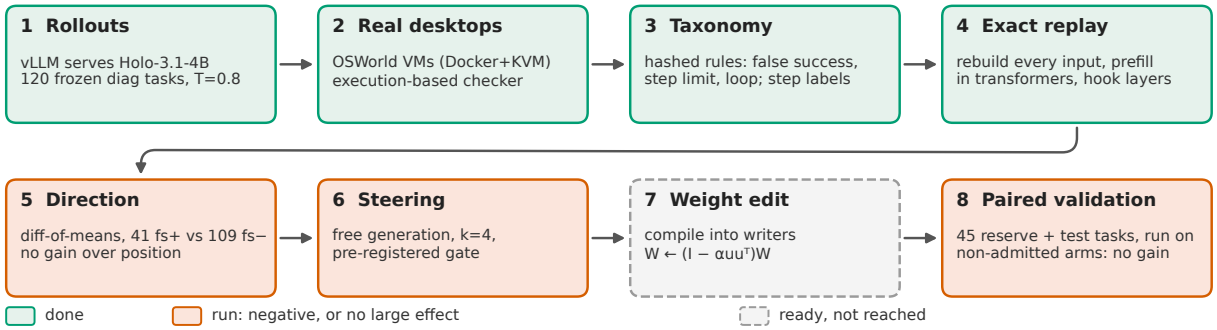


Figure 2: The pipeline and where it stands. Stages 1–4 are complete and verified. The official direction analysis (5), on 41 admissible FS+ steps, finds no signal beyond position. Steering (6) has been run, and no candidate shows a large effect. The compiled weight edit (7) is built and tested on null patches, but not reached because no candidate has been admitted. The paired online validation (8) was run anyway, on arms that had not been admitted (§6.6), and shows no gain beyond the run-to-run noise of the original. Counts as of 2026-11-07 21:56:45 UTC.

declared infeasibility (`task.{0,4}infeasible|infeasible|cannot be (completed|done)|not possible`) is broader than that of the official OSWorld adapter for Holo, which accepts only `task.{0,4}infeasible`; our absolute scores are therefore not strictly comparable with H Company's, while paired comparisons within our harness are. vLLM 0.23 serves the 4B with thinking enabled, $T = 0.8$, at most 50 steps and the three latest screenshots in context. OSWorld [38] tasks run in Ubuntu VMs under Docker+KVM, at most four in parallel, and are scored by the benchmark's execution-based checkers. We fixed three infrastructure issues in the VM provider (a memory cap, binding VM ports to localhost, an nftables network script) and modified no task or checker.

Frozen split. From the 361 tasks of RealtimeGUI's `test_nogdrive.json`, we sort ids by SHA-1 within each domain and deal them alternately into a DIAG and a TEST pool, keeping 120 tasks each with per-domain quotas. The split files have SHA-1 prefixes `51e670b3` (diag) and `957d68e7` (test). No construction or screening read the test split. Four online runs then used it (§6.6): the original, the ExPO edit, which went to it without having passed the gate, a partial corrective-prompt run, and the CUA-Lite merge under a decision rule fixed in advance.

MiniWoB++ harness (0.8B). Our first attempt used Holo-3.1-0.8B on MiniWoB++ [20, 27] through BrowserGym [5], with the task area cropped and upscaled 5×; it is described in §8 and Appendix F.

Hardware and licences. Rollouts, replays and screens ran on one NVIDIA H200 shared between the vLLM server and research jobs; edited checkpoints were also served on rented H100 GPUs, and the OSWorld VMs ran on a separate CPU host. As order-of-magnitude estimates, the project used about 14 hours of one H200 (rented, shared by every research job and the rollout server, so per-experiment GPU-hours are not separable), about 8 H100 GPU-hours (USD 31.9 billed over two accounts) and about 60 GitHub Actions CPU jobs for the OSWorld VMs. Holo-3.1-0.8B and -4B, Qwen3.5-4B, the CUA-Lite donor, OSWorld and BrowserGym are released under Apache-2.0 and MiniWoB++ under MIT; the RealtimeGUI repository has no licence file at the cited commit, and we use it unmodified except for the model provider.

3 Most failures are a false "done"

Taxonomy. Each episode gets one primary category from fixed rules over its actions, screenshots and final score. *False success*: the agent calls `answer` on a feasible task scored 0, or claims completion on an infeasible task. *Loop*: at least three consecutive identical actions while less than 1% of pixels change. *Step limit*: the 50-step budget is exhausted without a loop. At step level, FS+ marks the final `answer` of a false success and FS− that of a true one. A correctly worded refusal of an infeasible task is excluded from both. The rule file is hashed before any direction is fitted (current hash `1e5079afb500`); later changes are logged with their hash and none altered a label. Before fitting any direction we also require that the target category cover at least 25% of failures and provide at least 40 positive steps (the GO rule).

Result. On the diagnostic split (120 tasks, 119 scored), Holo-3.1-4B scores 56.7% (the official OSWorld score, partial credit included, the convention used for every score in this paper) and fully solves 66 tasks: 55.5%, 95%

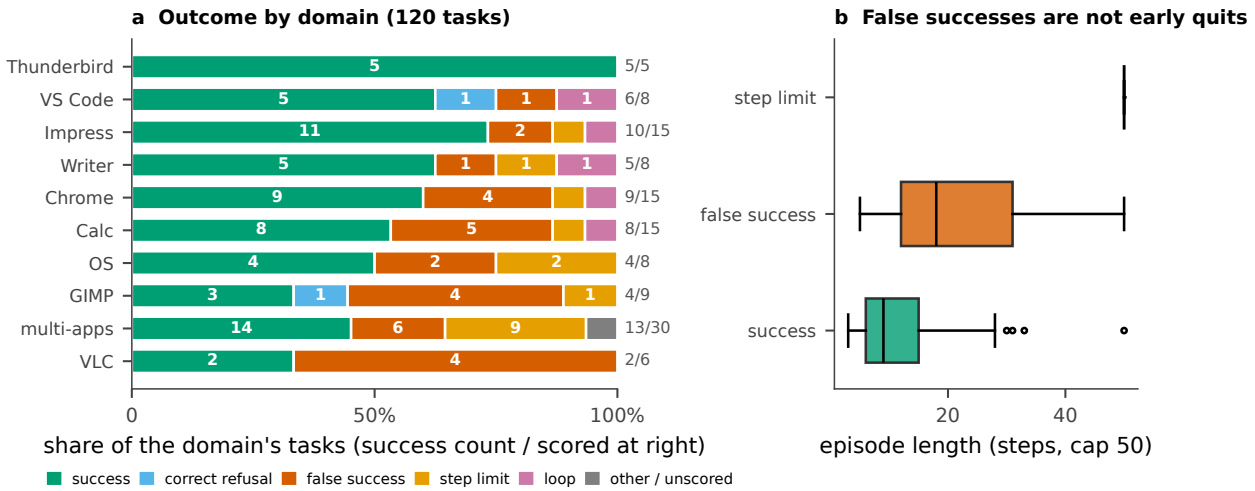


Figure 3: False success is spread across domains and is not an early quit. Holo-3.1-4B, OSWorld diagnostic split, 50 steps, $T = 0.8$. (a) Primary outcome per domain, sorted by success rate; numbers in bars are task counts, and the right column gives fully solved tasks over scored tasks (the *success* category also counts the 2 partially scored tasks). (b) Episode length by outcome. The full table is Appendix Table 8.

Wilson CI 46.5–64.1. An independent reproduction with the same public harness reports about 45.5% at 100 steps on a different task set [1]; we do not claim the two numbers are comparable. Of the 51 failures (Figure 3), 29 (57%) are false successes, 16 (31%) have the step limit as primary category (21, or 41%, exhaust the 50 steps at all, loops included), 5 (10%) loop, and 1 is unclassified; 1 further task is unscored (the "Other + unsc." column of Table 8 counts both). 9 false successes are claims of completion on infeasible tasks, and 20 are feasible tasks done wrongly or partially. On an infeasible task the correct action is `answer` with a refusal, so $P(\texttt{answer})$ is the wrong metric there; one such context, Chrome `480bcfea` at step 4, enters several screens below, and Appendix A gives every count with and without it. These are mostly not misworded refusals: among the 75 tasks open to construction, 6 are infeasible (11 episodes), and of the 10 final answers 8 claim success, while only 1 task states its inability, in words the harness rule misses. They are not early quits: the median false success ends after 18 steps. Every domain has false successes except Thunderbird, and Chrome, Calc, GIMP and VLC have four or more each (Fig. 3a). False success passes the share criterion of the GO rule, but its 27 FS+ steps alone do not pass the count criterion. With a second pass over the same tasks, stopped at 80 scored tasks when the GPU returned to production, 41 FS+ and 109 FS− steps are admissible, above the required 40 (as of 2026-11-07 21:56:45 UTC).

3.1 The model says "done" on false and true completions alike

We re-sampled the original model on six screening contexts from five tasks, two per stratum ($k = 4$, production sampling, thinking enabled, invalid outputs counted; Fig. 1c and Table 2, first row). It calls `answer` on 8/8 false-completion samples and 8/8 true ones, and never on mid-episode contexts (0/8). It reproduces the terminal versus non-terminal distinction exactly; the false versus true distinction is invisible. Any repair must therefore change a decision the model takes with near certainty in both cases, and only in one of them.

The reasoning has the same shape in both cases. On 16 re-sampled reasoning traces at completion steps of 4 construction tasks, 8 on false and 8 on true completions, every trace opens with a success claim ("The task has been completed successfully", "The document has been saved successfully") at generated-token index 4–5, counted from zero, with no verification word before it (0/16), then lists the steps taken (median 116 tokens). Because true completions show it too, this is a property of Holo's output format at completion steps, not a marker of false success. It locates the written conclusion, not the moment the decision forms inside the model. A direction fitted on these tokens separates the two pairs perfectly, but so do context length and a single lexical cue, so we give it no weight; steering along it is negative (Table 2). What the format skips is the check: the Spotify episode states that the installation cannot finish and claims success one step later (Fig. 1a).

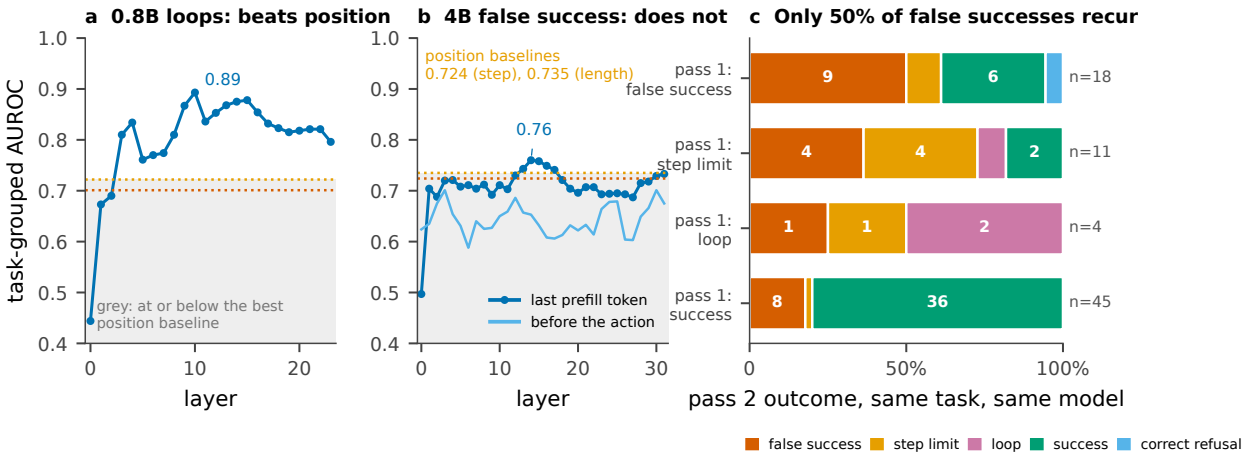


Figure 4: Two of the three traps. (a) Holo-3.1-0.8B, MiniWoB++: a loop direction separates loop steps far above the step-index (red) and context-length (orange) baselines, with task-grouped cross-validation. It still fails every causal test (§8). (b) Holo-3.1-4B, OSWorld, official analysis on both passes (41 FS+, 109 FS−, 95 tasks): the false-versus-true completion direction barely clears the position baselines at its best layer, and not once position is controlled (Table 1). The grey band marks AUROCs at or below the best baseline. (c) The unmodified 4B run twice on the same tasks (80 tasks scored in both passes). Only 9 of 18 pass-1 false successes recur, against 36 of 45 pass-1 successes.

4 From a served agent to activations

Rollouts are served by vLLM [17], which does not expose activations. We therefore never record activations during rollouts. For each recorded step we *rebuild* the exact chat the agent sent: system prompt, task, `<observation>` blocks, at most three screenshots with older ones replaced by `[screenshot evicted]`, parsed tool calls and tool outputs. We render it with the model's chat template and run one forward pass in `transformers`, hooking the residual stream and MLP output of every layer at two positions: the last prefill token, and the token before the action JSON.

Fidelity is tested, not assumed. (a) *Messages.* Running the real agent loop with a fake client and environment, the rebuilt messages are identical to those sent on 7/7 steps, also when three consecutive parse failures are injected before steps 1, 4 or 5, the harness's image-eviction edge case. (b) *Tokens.* The prefill length in `transformers` equals vLLM's `prompt_tokens` on 12/12 steps of a real trajectory (4,459 to 10,432 tokens), and the replay loop repeats this check on the first and last step of every task. (c) *Likelihood.* The replayed model gives its own recorded actions a mean log-probability of -0.16 per token, against -1.10 for synthetic actions. One divergence is invisible in the files: an exception in `env.step` leaves an unrecorded assistant message in the live context. The diagnostic pass logged 0 such events and 6 triple parse failures, which test (a) shows to be harmless.

5 Three traps that would each have produced a positive result

Trap 1: the direction measures position. False completions come later in an episode than true ones, so anything that grows with the episode separates them. A first difference-of-means direction between FS+ and FS− steps (22 and 55 steps), cross-validated by task, peaked at AUROC 0.836 against 0.870 for the step index and 0.880 for context length. Reported without baselines, 0.836 reads as a solid detector; in fact it was worse than counting steps. The official analysis, run once FS+ steps passed the pre-registered 40 (41 FS+, 109 FS−, 95 tasks, both passes; exploratory because the second pass stopped early), confirms it (Table 1, Fig. 4b). The best direction, at layer 14 on the last prefill token, reaches 0.76 (logistic probe 0.71) against 0.724 and 0.735 for the two position baselines, and the token before the action does worse (0.70). Controlling for position directly, a logistic model on step and log context length scores 0.722; adding the best direction raises it by at most 0.04, at a layer chosen after the fact, within noise. We find no false-success signal in the residual stream beyond the length of the episode, which is consistent with the failure of every direction-based intervention below. A value-axis direction in the style of Jiang et al. [13] shows the same confound in a separate construction (appendix E). Our protocol therefore matches positives and negatives on step and context length before fitting, and reports both baselines next to every AUROC.

Trap 2: regression to the mean. Agent outcomes are stochastic at $T = 0.8$. Selecting tasks because they failed in one run selects partly for bad luck. Re-running the unmodified agent on the same tasks, only 9/18 (50%, 95% CI

Predictor (task-grouped 5-fold CV)	Best layer	AUROC
Step index alone	–	0.724
Context length alone	–	0.735
Direction, last prefill token	14	0.760
Direction, before the action	3	0.701
Position (step, log length), logistic	–	0.722
+ direction, last prefill token	15†	0.762
+ direction, before the action	22†	0.717

Table 1: Official false-success direction analysis: no signal beyond position. Holo-3.1-4B, 41 FS+ and 109 FS− steps from 95 tasks, two passes, 5-fold cross-validation grouped by task, labels frozen before analysis. Exploratory: the second pass stopped at 80 tasks. †Layer chosen after the fact.

29–71) pass-1 false successes are false successes again (an earlier snapshot of the second pass, on fewer tasks, gave 42%), and 6 of them now succeed (Fig. 4c). A patch evaluated as "false-success tasks before versus after" would therefore be credited with about 50 points of reduction while doing nothing. Our validation compares two fresh runs, original and patched, on a frozen reserve of `confirm_tasks.json` tasks that no intervention was built on (§5.1).

Trap 3: detecting is not causing. The 0.8B loop direction is a textbook detection result (Fig. 4a): task-grouped AUROC 0.893 at layer 10, against 0.70 for the step index and 0.72 for context length, replicated at 0.904 on an independent refit. It fails every causal test we ran (§8). Projecting it out of every writer erases the marker from the residual stream (the loop/non-loop projection gap at layer 10 falls from 0.1653 to -0.00008) and leaves loop actions as likely as before relative to healthy ones. The marker can be deleted without touching the behaviour it marks.

5.1 The protocol these traps impose

1. **Free generation is the only efficacy measure.** Teacher-forced log-probabilities of a recorded action mix format, arguments and end-of-sequence, and on the 4B the decision is taken during reasoning, after which $P(\texttt{answer})$ is close to 1 for true and false completions alike. Each candidate regenerates the full reasoning ($k = 4$ per context, fixed seeds shared across arms, production sampling with `top_k=20`), and invalid or truncated outputs stay in every denominator.
2. **A gate fixed before any data.** On FS+ contexts of tasks not used to build the intervention, $P(\texttt{answer})$ must fall by at least 25 points and 50% relative. The FS− answer rate and the rate of valid non-answer actions on mid-episode contexts may each fall by at most 5 points, and invalid outputs may rise by at most 5 points. The FS− criterion is what separates "less false success" from "less success".
3. **Controls.** A random direction with the same norm and scope, an identity hook that measures the noise floor, and a fixed budget of generation batches per hypothesis, counted before execution, failures included. Two consecutive settings without progress stop a hypothesis.
4. **Disjoint confirmation.** The official direction analysis is gated at 40 admissible FS+ steps. The `confirm_tasks.json` reserve is excluded from every construction and screening, and was to be read only to confirm a committed candidate, with both arms re-run fresh. In the event no candidate was admitted, and five non-admitted arms were run on it (§6.6); we list this as a limitation (§11).

6 No large effect found

6.1 Where the tool choice is carried

Before steering, we mapped where the choice of `answer` is made. On two FS+ and two FS− completion steps, we patched the residual stream from the same task's previous step (whose recorded action is a click) at each of 32 layers and four positions, 512 patches in all, with the original reasoning held fixed. The quantity is the logit margin between the first token of `answer` and that of the best other tool. Identity patches give exactly zero.

The decision is invisible at the end of the prefill and barely visible mid-reasoning: the largest mean margin change is -0.19 and -2.28 logits there, for FS+. It appears only at the tool-name position, from about layer 16 upward, where patching moves the margin by up to -34.8 logits on FS+ and -37.5 on FS− (Fig. 7). Component scans single out three attention heads, L19/H15, L23/H1 and L31/H14, and ten late MLP neurons. Their attention falls mostly on the system-prompt definition of `answer` (L19/H15, L23/H1) and on the JSON being written (L31/H14). Removing all of them together lowers the margin by 11.2–12.2 logits, and flips 0 of the 4 decisions (smallest remaining margin 2.9).

Across all 992 component patches and 28 joint patches, 0 and 0 decisions flip: the components move the margin, not the action. The only flips come from residual patches (21 on FS+ and 23 on FS− of 512), at the tool name in the last layers, where replacing the residual stream transplants the donor's choice into the output head. They show where the choice is carried, not where a false judgement is formed. The effect is the same on true and false completions (Fig. 7c). It is a generic tool-retrieval circuit, not a false-success circuit.

A ceiling test. The patches above replace one layer at a time; a ceiling test asks how much must be replaced at once. With the recorded input forced and only logits read, the outputs of whole blocks of writers are swapped in at the decision position from a donor step where the model clicked: the same task's previous step for two FS+ contexts, another task's step for one true completion (Fig. 6). On the first FS+ context (margin +16.9), all 32 MLPs together bring the margin to +0.9 without flipping it, while the MLPs and token mixers (attention and DeltaNet) of layers 12–23 flip it (−3.5), as do those of layers 24–31 (−10.0), and those of layers 0–11 do not (+16.1). The second FS+ context behaves the same (MLPs +2.1, block 12–23 −3.0), and so does the true completion (block 12–23: from +16.4 to −2.2). Swapping with itself and swapping the final residual reproduce the baseline and the donor exactly. Three caveats bound this result. The first FS+ context is an infeasible task, on which `answer` (with a refusal) is the correct tool (Appendix A). The donor differs in screen, reasoning, thought and position, not only in decision. And since a true completion flips too, the test locates where the identity of the next tool token is assembled, not a mechanism of false success. No particular block is needed: the MLPs and mixers of layers 12–23 or of layers 24–31 each flip the first context, and removing a single direction of the swap difference from the writers of layers 12–23 brings the margins of the two FS+ contexts to +1.4 and +1.6, but also those of two true completions to +4.8 and +2.8. It is consistent with the failure of every edit confined to one or two layers or to the MLPs, and it says nothing about which edit would separate false from true completions.

6.2 The tool call leans on the text already written

The tool call leans on the text the model has already written, for true completions as for false ones. On the 19 false and 20 true completions of the strictly feasible cohort (§6.5), teacher-forced with the screen, history, reasoning and token count fixed, replacing the note and thought by a neutral text of the same token length ("Observing the current screen.") removes 14.0 logits of `answer` margin on false completions (95% CI 12.9–15.1; original 17.3) and 14.5 on true ones (12.9–16.0; original 18.0), 82% and 80% of the margin per task. The two losses differ by −0.46 logits (−2.36 to +1.50; label permutation $p = 0.65$). A counterfactual text of the same length ("The task is not complete yet; I still need to inspect the screen.") leaves 0.10 and 0.61 logits, nearly cancelling the margin on average (`answer` still leads in 10/19 and 13/20 contexts), while neutralising the recorded reasoning instead removes only 4.1 and 4.8. One true completion has an empty note; without it, the true-completion loss is 15.0. The written note and thought thus carry about 80% of the `answer` margin in both strata, yet without them `answer` remains the preferred tool in 37 of 39 contexts (18/19 false, 19/20 true): the conclusion is reinforced by the note, not created by it. This is an attenuation of the margin, not a change of decision. Nor does a probe find a robust discriminator in the residual stream of these contexts: of 72 out-of-fold probes (6 layers, three positions, four models), the best, chosen after the fact, reaches AUROC 0.64 (0.47–0.80) against 0.70 for the raw step index, and its permutation p of 0.04 becomes 1.0 after correction for the 72 probes. With these sample sizes a pre-specified score would need an AUROC of at least 0.725 (one-sided) to 0.755 (two-sided) to be detected with 80% power, so the absence of the information is not proven. These are artificial inputs, and the joint replacement does not separate the wording of the conclusion from the details the note describes.

A first version of the test, on two FS+ contexts, illustrates it with a donor's text rather than a neutral one: it keeps the screen, the history and the weights, and replaces only the text the model has generated at the decision step with that of the same task's previous step, where the model clicked. On the first FS+ context (margin +16.9), the donor's reasoning alone lowers the margin to +7.6, the donor's JSON prefix alone (note and thought) to +1.1, and both together flip it (−7.4). The second context behaves the same (+18.4; reasoning +11.1, JSON prefix −3.1, both −7.6). Three limits weighed on that version, the first of which the cohort test above removes: it had no control on true completions or with a neutral text. Only its second context is feasible, and the donor is the previous step, whose text announced a click, so that copying its text largely copies its decision. And the text does not account for everything: with all of it swapped, the margin stays 9.8 and 14.5 logits above the donor's, a share that comes from the screen and the history. Within these limits, the tool call depends causally on the text already written, the note and thought weighing most, and the 13.9–19.1-logit margin of §6.3 is largely conditioned on that text rather than an intrinsic robustness of the decision. An attribution on the same cohort locates the channel. Replacing the inner tokens of the note and thought with spaces, at the same length and positions and with the screen, history and reasoning kept, lowers the teacher-forced margin by 7.55 logits on false completions (mean 17.3 before) and by 8.18 on true ones (mean 18.1 before, measured on its own cached replay path), less than the neutral text above, and lowers it on every context of both (19/19 and 20/20); the difference between the two losses is −0.64 logits (95% CI −2.13 to +0.80). The note and thought carry the decision

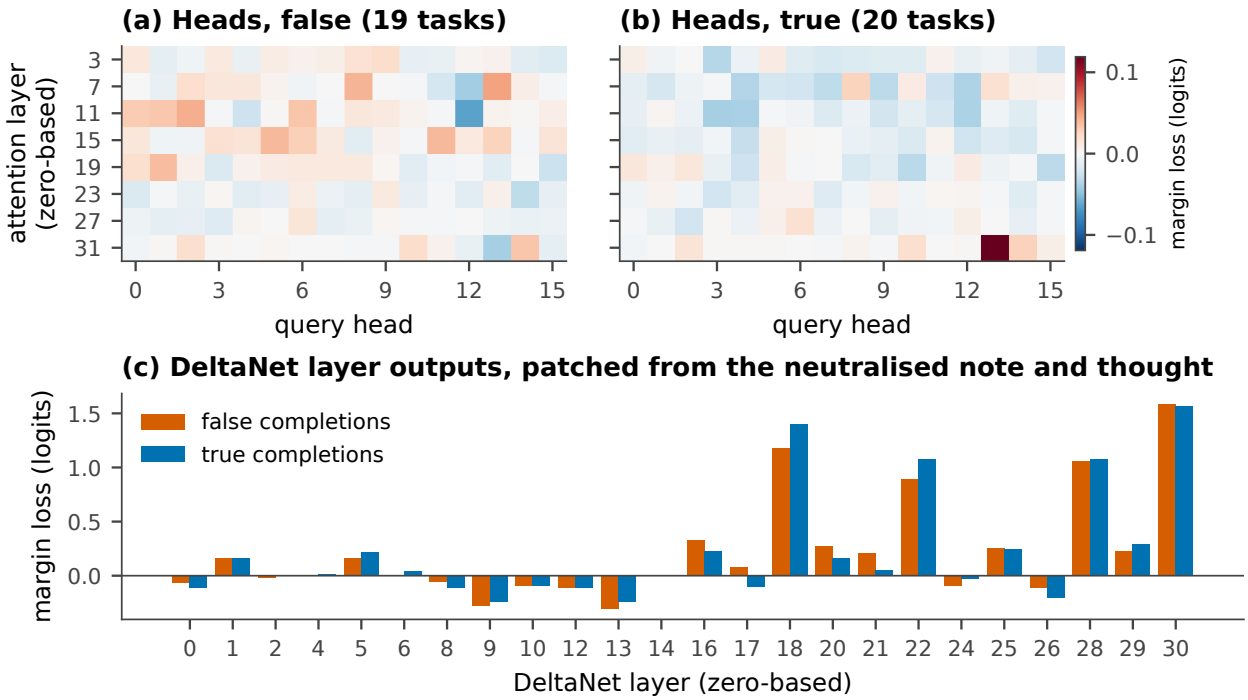


Figure 5: The written conclusion reaches the tool choice through the DeltaNet memory, for true completions as for false ones. Teacher-forced `answer` margin lost when the note and thought are neutralised, on the strictly feasible cohort. (a, b) Blocking the final query's attention to the note and thought, one head at a time: at most 0.12 logits (colour scale). (c) Replacing one DeltaNet layer's output at the decision position by its value under the neutralised note and thought: 0.9–1.6 logits at layers 18, 22, 28 and 30, alike for both strata (note the ten times larger scale). Single-layer patches do not add up to the whole effect (7.55 and 8.18 logits). Heads and layers are screened on the same cohort: exploratory.

as much for true as for false completions: the channel is real, but not selective. It runs chiefly through the recurrent memory rather than attention (Fig. 5). Replacing the output of a single DeltaNet layer at the decision position by its value under the neutralised text costs 0.9–1.6 logits at layers 18, 22, 28 and 30 (zero-based) and at most 0.33 at the other layers. Blocking the final query's attention to these tokens, one head at a time over the 128 heads of the eight full-attention layers, costs at most 0.048 logits on false completions and 0.12 on true ones, and at best 0.05 logits more on false than on true. In this hybrid model, the conclusion already written is carried forward by the recurrent memory. The same contexts define the comparison and the cohort, so this screen is exploratory, and layer-output patches locate carriers of the change, not its mechanism.

6.3 Every steering arm

Table 2 lists every intervention run on the 4B false-success target, each under the gate of §5.1 and on the same six screening contexts. Each line corresponds to a distinct hypothesis taken from the literature.

- *Tool choice is linearly steerable* [36]: add the position-matched FS+−FS− direction at the last prefill token, at layer 16 ($\alpha = -1.20$ times its norm) and layer 24 ($\alpha = -5.54$).
- *The decision is taken during reasoning* (§3.1): add a "conclusion" direction, fitted on the reasoning tokens, at every reasoning token of layer 31.
- *False success is excess expected value* [13, 15]: push toward low value or low commitment at layer 24 or 18.
- *Late MLP neurons write the decision* [7]: zero the top-16 or top-64 neurons by direct attribution to the `answer` logit.
- *The model fails to check*: add a "verify" versus "conclude" direction (contrastive activation addition) at layer 18 at 1 and 2 times the residual norm, over the prefill and the first 20 generated tokens.
- *False success is a perception of completion*: subtract the imminent-completion direction of §6.4 at layer 15.
- *The model opens with its conclusion* (§3.1): raise the output row of the token "Wait" for the states that open a completion, a 2560-weight edit of the output head.
- *Memory of the early episode fades*: lengthen or shorten the retention of the nine late linear-attention layers by a factor 1.25.

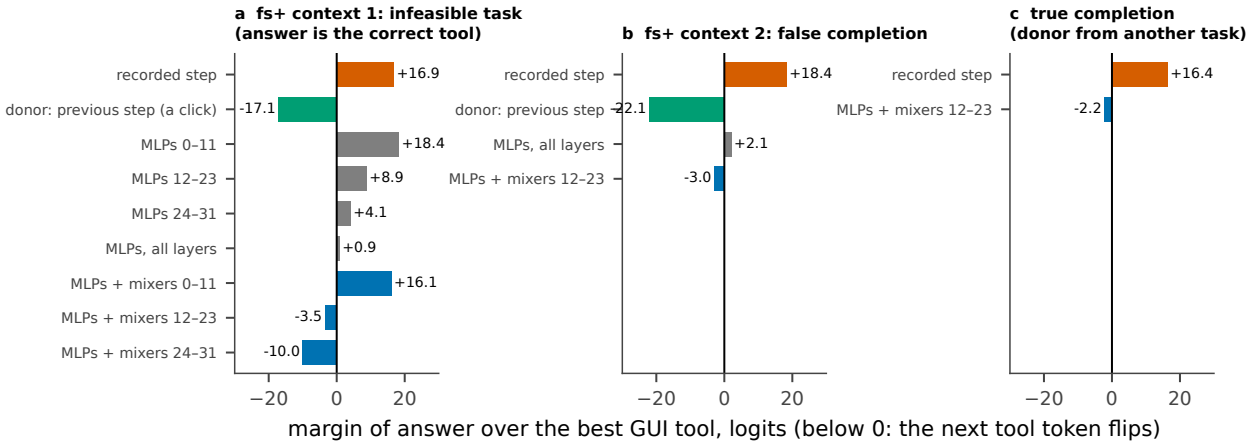


Figure 6: Ceiling test: the next tool token flips only when MLPs and token mixers of a whole block are swapped, on false and true completions alike. Holo-3.1-4B, recorded input forced, logits only. Bars give the margin of `answer` over the best GUI tool after swapping in, at the decision position, the outputs of the indicated writers from a donor step where the model clicked. (a) The first FS+ context is an infeasible task, where `answer` is the correct tool. (c) The true completion uses a donor from another task. Three contexts; exploratory.

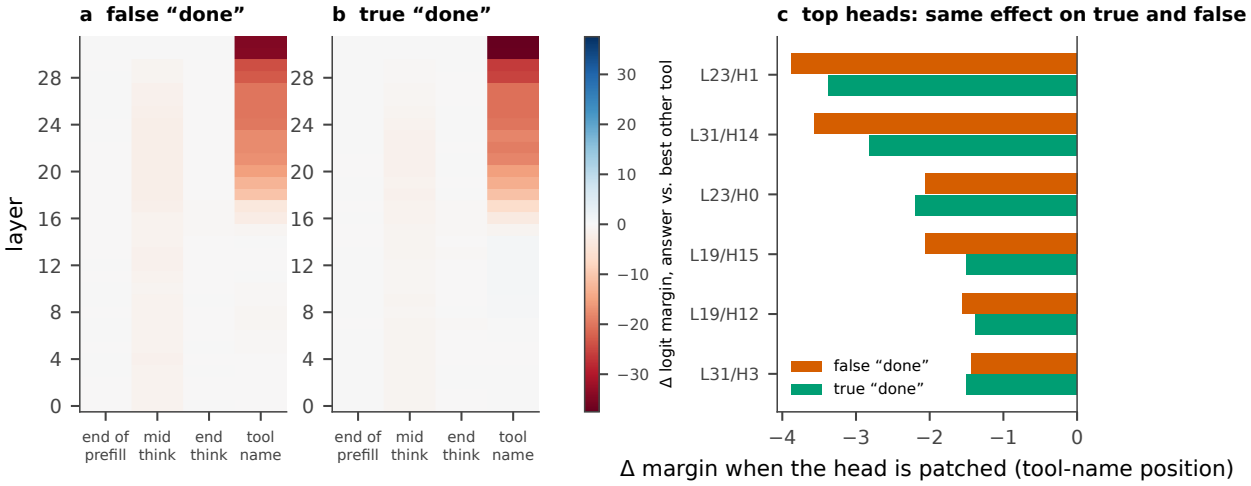


Figure 7: The tool choice becomes visible late, at the tool name, by the same components for true and false completions. Activation patching on Holo-3.1-4B from the previous step of the same task (two tasks per class; exploratory construction data). (a, b) Mean change of the `answer`-versus-best-other-tool logit margin when the residual stream is patched at one layer and position; red lowers `answer`. (c) The six heads with the largest effect at the tool-name position; red bars are false completions, green bars true ones.

- *The circuit of §6.1 carries it*: scale its three heads and ten neurons (C1) by $1 - \alpha$ with $\alpha = 0.5$ and 1, against a random direction of the same contribution norm.

No arm passes the gate, and none shows a large effect. Most leave the false-“done” rate at its original value, including removing the whole C1 circuit ($\alpha = 1$), which answers on every false completion, like the random control. The neuron arms, screened on six other disjoint contexts whose own baseline is 6/8 FS+ answers, were stopped by a mathematical bound: with 4/4 answers already observed on the first unseen FS+ task, at most 3 answers out of 8 would have been admissible. The value-axis arm did change the text. An independent reader found an explicit intention to re-evaluate in 3/8 FS+ reasoning traces against 1/8 at baseline, but 0 verification actions and the same answer rate. The model wrote that it should check, then answered anyway.

The “Wait” row is the one edit that lowers false answers without touching anything else: 6/8 on false completions, with 8/8 true completions kept, 8/8 healthy actions valid and 3/3 general-text controls token-identical. It still fails the 50% relative criterion, and an independent review rates none of the two replacement actions useful: one reloads a settings page for an invented reason, the other scrolls without fixing the dates. Making the model pause is not enough

to make it repair. A conditional variant adds an early end-of-reasoning row to the same edit (5101 weights). Inside its construction set, 5/8 false completions still answer. On one Budget sample the model reasons correctly that next month is December, says the dates must be changed, and generates a click on the date summary of the results page; it was not executed. These are intentions to correct, not corrections: an independent review of the screenshot finds that the dates sit in plain summary text, next to a separate "Modify Rental Details" control, and that no step proves the summary opens an editor. All 7 such generated clicks, over 4 lots of this edit and its diagnostics, are reclassified as unverified, 0 confirmed useful; no effect on the interface or on the OSWorld score is shown. Nor does the edit transfer. On nine training tasks left out of its construction, the final audit counts 0/12 useful actions on false completions (original 0/12), 3/12 outputs that name the real error (original 1/12), and 7/12 false finishes (original 8/12), while true completions go from 10/12 to 9/12. The edit leaves a mechanistic signal, the model recognising the error then rationalising it away, and no demonstrated useful effect.

These screens are pilots. Each efficacy cell holds 8 generations from two tasks. 8/8 has a Wilson 95% interval of [68, 100]%, and samples clustered in two tasks make the real uncertainty larger, so a rejection means only that no large effect appears on these contexts, not that none exists. By design the gate flags a reduction to at most 4/8 answers; smaller effects pass unnoticed. Table 2 gives the interval of every efficacy cell.

A ceiling effect is likely. At the tool-name position the margin of `answer` over the best other tool is 13.9–19.1 logits on false completions and 10.9–17.8 on true ones. This position comes after the JSON `thought` field, which the output schema writes before the tool call and which already states that the task is complete, so the margin measures how confidently the model carries forward a decision it has already verbalised, not the margin of the decision itself. Even so, the perturbations tested are small: next to the residual stream, they are 0.13 (layer 16) and 0.23 (layer 24) of the mean residual norm for the matched direction, 0.25 for the reasoning-token direction (difference-of-means norm 24.5 against a residual norm of 50.0 at layer 31), and 0.5 for the value axes, while removing the whole C1 circuit lowers the margin by 11.2–12.2 logits without flipping it. Larger doses do not break the ceiling either. The "verify" direction at one and two times the residual norm leaves FS+ at 8/8; at dose 2 a healthy context loops on "I see" up to the 3,072-token cap, which stopped the arm after 23 of 24 outputs. Dose sweeps at layers 24 and 31 with random controls were not run.

A second target, getting stuck at the step limit, shows the opposite failure of steering. A residual addition at layer 16 or 24 damaged healthy actions before any stuck context was generated; after 5/24 outputs, the best reachable health rate was 91.7%, below the required 95%.

6.4 A detector of imminent completion beats position, not false success

All directions above contrast false and true completions, which differ in position. `astra-perceive` sidesteps this by learning a direction *inside successful episodes*: the last-prefill residual of the final, correct `answer` step minus that of the step just before it, on 58 pairs from 40 construction tasks (reserve excluded, cross-validation grouped by task across both passes). Because both members of a pair sit next to each other in the same episode, position cannot separate them across tasks. The direction peaks at layer 15 with task-grouped AUROC 0.910, against 0.552 for the step index and 0.555 for context length (Fig. 8a). This is the first signal in this study that position does not explain, in contrast to the false-success direction of Table 1. Within a pair, both baselines are trivially perfect, so the pooled AUROC is the relevant comparison, and the layer was chosen on the same cross-validation.

Read on 14 false/true completion pairs matched on pass, step and context length, with each task's direction fitted without that task, the direction rises at the "done" step in 14/14 false and 14/14 true completions (mean 1.11 and 1.54 standard deviations; Fig. 8b). At the terminal step, false completions sit -0.14 standard deviations from true ones, above them in only 6/14 pairs. It is therefore a detector of imminent completion, not of false success: it fires before a false "done" as it does before a true one. On these 14 pairs, the projection reads a false completion as complete. This is compatible with a perception error: the screen is taken as showing a finished task. Two same-task image pairs from construction data illustrate it, a spreadsheet cell claimed green whose text has no green pixels, and rental dates claimed for next month while the page shows the current one. It does not separate misperception from commitment to the action. Subtracting it at layer 15 (up to 0.68 of the residual norm, on its own screening contexts whose reference is 6/8 FS+ answers) gives 5/4/5/5 FS+ answers out of 8 for $\alpha = 0.5, 1, 2, 4$, against 7/8 for a random direction of the same norm, and no dose passes the gate (Table 2). The few drops are not repairs: the non-answer actions that replace `answer` are empty or invented text entries, such as a made-up e-mail address. Attention maps on two same-task pairs found no head, out of 80 tested, that reads the evidence region more when the model succeeds (0/80 pass the frozen rule). The perception hypothesis is not refuted by this small screen, but it has no support beyond the projection of Fig. 8.

Hypothesis & intervention	Where	Dose	Out.	fs+ `answer` [95% CI] ↓	fs− `answer` =	healthy valid =	invalid	Verdict
Original (reference)	–	–	–	8/8 [68, 100]	8/8	8/8	0/24	–
`4b-x`: residual addition, matched fs direction	L16, last prefill, $\alpha = -1.20$	0.13	24	8/8 [68, 100]	8/8	8/8	0/24	rejected
`4b-x`: residual addition, matched fs direction	L24, last prefill, $\alpha = -5.54$	0.23	24	8/8 [68, 100]	8/8	8/8	0/24	rejected
`think`: addition of the "conclusion" direction	L31, every reasoning token, $\alpha = -0.5$	0.25	24	8/8 [68, 100]	8/8	8/8	0/24	rejected
`value`: push toward low value (Value Axis)	L24, last prefill, $\alpha = 0.5$	0.50	24	8/8 [68, 100]	8/8	8/8	0/24	rejected
`value`: push toward low commitment (Value Axis)	L24, last prefill, $\alpha = 0.5$	0.50	24	8/8 [68, 100]	8/8	8/8	0/24	rejected
`value`: push toward low value (Value Axis)	L18, last prefill, $\alpha = 0.5$	0.50	24	8/8 [68, 100]	8/8	8/8	0/24	rejected
`value`: push toward low commitment (Value Axis)	L18, last prefill, $\alpha = 0.5$	0.50	24	8/8 [68, 100]	8/8	8/8	0/24	rejected
`neurons`: zero the top-16 MLP neurons writing `answer`	L20–31, whole generation	–	4	4/4 [51, 100]	–	–	0/4	rejected (bound)
`neurons`: zero the top-64 MLP neurons writing `answer`	L20–31, whole generation	–	4	4/4 [51, 100]	–	–	0/4	rejected (bound)
`circuit`: scale C1 (3 heads + 10 neurons) by $1 - \alpha$	L19/23/31 heads, L30–31 neurons, $\alpha = 0.5$	–	24	8/8 [68, 100]	8/8	8/8	0/24	rejected
`circuit`: scale C1 (3 heads + 10 neurons) by $1 - \alpha$	L19/23/31 heads, L30–31 neurons, $\alpha = 1.0$	–	24	8/8 [68, 100]	8/8	8/8	0/24	rejected
`circuit`: random control: same-norm Gaussian direction removed	L19/23/31 heads, L30–31 neurons, $\alpha = 0.5$	–	24	8/8 [68, 100]	8/8	8/8	0/24	control
`perceive`: subtract imminent-completion dir.	L15, last prefill, $\alpha = 0.5$	0.09	24	5/8 [31, 86]	8/8	7/8	0/24	rejected
`perceive`: subtract imminent-completion dir.	L15, last prefill, $\alpha = 1$	0.17	24	4/8 [22, 78]	8/8	5/8	0/24	rejected
`perceive`: subtract imminent-completion dir.	L15, last prefill, $\alpha = 2$	0.34	24	5/8 [31, 86]	8/8	7/8	0/24	rejected
`perceive`: subtract imminent-completion dir.	L15, last prefill, $\alpha = 4$	0.68	24	5/8 [31, 86]	8/8	5/8	0/24	rejected
`perceive`: random control, same norm	L15, last prefill, $\alpha = 4$	0.68	24	7/8 [53, 98]	8/8	6/8	0/24	control
`verify`: add a "verify" vs. "conclude" direction (CAA)	L18, prefill + 20 first tokens, dose 1	1.00	24	8/8 [68, 100]	8/8	8/8	0/24	rejected
`verify`: add a "verify" vs. "conclude" direction (CAA)	L18, prefill + 20 first tokens, dose 2	2.00	22	8/8 [68, 100]	8/8	6/6	0/22	rejected (bound)
`verify`: random control, same norm	L18, prefill + 20 first tokens, dose 1	1.00	24	8/8 [68, 100]	8/8	8/8	0/24	control
`verify`: random control, same norm	L18, prefill + 20 first tokens, dose 2	2.00	20	8/8 [68, 100]	7/8	4/4	0/20	control
`mit1`: raise the "Wait" output row for completion openings (weight edit)	`lm_head`, 1 row, 2,560 weights	–	24	6/8 [41, 93]	8/8	8/8	0/24	rejected
`mit1`: shorten memory of late linear-attention layers (weight edit)	`A_log`, 9 layers, retention $\times 0.8$	–	24	8/8 [68, 100]	8/8	8/8	0/24	rejected
`mit1`: lengthen memory of late linear-attention layers (weight edit)	`A_log`, 9 layers, retention $\times 1.25$	–	24	8/8 [68, 100]	8/8	8/8	0/24	rejected

Table 2: Every causal test on the 4B false-success target. Of the two FS+ contexts of most arms, one is an infeasible task where `answer` is correct, so feasible false successes are represented by a single context and 4 samples per arm; both FS+ contexts of the `perceive` arms are infeasible (Appendix Table 5 gives every count without them). Free generation, $k = 4$, seeds shared across arms, six screening contexts from five tasks (two per stratum). "fs+ answer" must fall (gate: -25 points and -50% relative); "fs− answer" and "healthy valid" must stay within 5 points. "Out." is the number of outputs generated out of 24 planned. "Dose": perturbation norm over the mean residual norm at that layer (nominal; – for neuron and head scalings). Brackets: Wilson 95% interval of the fs+ answer rate, ignoring clustering by task. Every screen is a pilot that can only detect large effects, a fall to 4/8 FS+ answers or fewer. The neuron arms use a different set of six contexts (their own original: 6/8 FS+ answers). *Rejected (bound)*: rejected before completion because the gate had become unreachable. *Control*: random direction, not a candidate. The `perceive` arms use their own screening contexts (reference FS+ 6/8, healthy 6/8). The `verify` dose-2 arm was stopped after 23 outputs when a healthy context looped to the token cap; its counts are from the last snapshot before that output. An L28 arm was not run.

6.5 Weight edits: a lever without a discriminator

Weight edits that need no direction, or that target a mechanism, are detailed in Appendix B; we summarise them here. Extrapolating the fine-tune away from Qwen, $W(\lambda) = W_{\text{Qwen}} + \lambda\,(W_{\text{Holo}} - W_{\text{Qwen}})$, called ExPO [44], leaves false answers at 16/16 in every arm but one (15/16, MLPs only, where true completions drop more); the edit at $\lambda = 1.25$ keeps text and JSON intact and runs online (§6.6). The mechanism-level edits H2–H10 (memory gates, MLPs or all language tensors moved toward Qwen, the end-of-thinking embedding, the attention temperature, the rotary share of the query gain) leave the tool call in place, break generation, or move the call only on the infeasible context, where theme clicks replace an answer that should have explained the impossibility (Table 7); two-neuron circuits, a narrow MLP edit adapted from weight-space programming and a corrective instruction injected into the reasoning do no better. Two edits do move the decision on feasible contexts, and neither shows an established selectivity.

Merging in a donor trained on desktop tasks moves the decision globally, not selectively. Adding a fraction β of the task vector of CUA-Lite, a Qwen3.5-4B fine-tune on ScaleCUA desktop data, to the language layers of Holo ($W_{\text{Holo}} + \beta\,(W_{\text{CUA}} - W_{\text{Qwen}})$; vision tower unchanged; input embeddings merged, the output head being tied to them

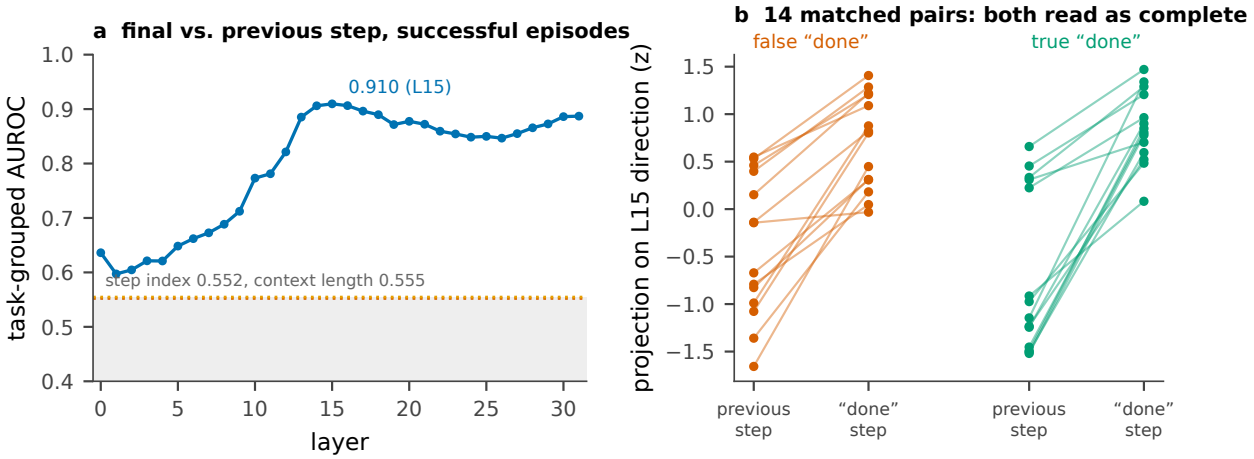


Figure 8: An imminent-completion direction beats position but does not separate false from true completions. Exploratory, no causal test. (a) Task-grouped AUROC of a direction learned inside successful episodes (final `answer` step vs. the step before it, 58 pairs, 40 tasks), by layer; the grey band is at or below the position baselines. (b) Cross-fitted projections at layer 15 on 14 matched pairs: the projection rises at the "done" step for false and true completions alike.

at load time), and serving it with the same 51 calls as the exact original model, re-served on the same GPU type and vLLM version (in a separate account and container), lowers false-completion answers to 15/16 at $\beta = 0.25$ and 11/16 at $\beta = 0.5$, and true-completion answers further, to 11/16 and 2/16 (original 16/16 and 16/16), with every healthy action valid and no invalid output; on the feasible FS+ context alone, $\beta = 0.5$ answers 3/8. The dose–response is a global drop of `answer`, not a discrimination. An independent audit finds the replacing actions coherent and unnecessary: scrolls after the model has recognised success, and verification clicks after true completions. The model continues instead of concluding, whatever the truth of the completion. A donor trained on filtered web tasks, Fara1.5-4B, leaves the decision in place at $\beta = 0.5$ (16/16 and 14/16). BF16 rounding blurs small merges: at $\beta = 0.5$, 88 of the 426 language tensors are left unchanged and the update norm is 2.34 against an intended 2.01; at $\beta = 0.25$, 101 tensors are unchanged. CUA-Lite's training data may overlap OSWorld tasks. Online, the merge shows no detectable gain over an original served by the same endpoint: +1.9 points over 165 tasks (95% CI −4.3 to +8.0; §6.6).

A closed-form edit of the writers that the ceiling test points to moves the decision as well, unselectively on the screening contexts; on a larger feasible cohort its selectivity is not established either way (below). In the spirit of AlphaEdit [8], the 24 residual writers of layers 12–23 (MLP down-projections and mixer output projections) receive a ridge update that maps FS+ keys (constructed on the ceiling-test contexts, the first of them infeasible) toward the donor displacement while being projected off the span of protected keys (true completions, healthy steps and text prompts). On contexts outside its construction, served next to the original on the same GPU, false-completion answers fall from 15/16 to 8/16 but true-completion answers fall further, from 15/16 to 3/16, with 16/16 healthy actions valid and 0 invalid outputs. The whole FS+ drop comes from one of the two contexts (7/8 to 0/8; the other stays at 8/8). A first visual audit of the replacing actions, which had described the screenshot of the previous step, was retracted; the corrected audit finds that on the e-mail true completion all four recipients were already complete, so the three generated `write` continuations there would potentially damage a correct state (they were not executed). A random update on the same writers, with the same right factors and its per-writer BF16 perturbation norm matched to the learned one within 0.1%, leaves the screening contexts in place (FS+ 16/16, true completions 16/16, healthy 16/16): the effect is specific to the learned direction, a real causal lever on stopping, without selectivity on these contexts. At half the dose the edit gives 12/16 and 9/16: at both doses, true completions fall on these contexts as fast as or faster than false ones. A 25% dose was not run.

Two independent methods, a merge and a closed-form edit, thus move the decision. On the screening contexts, true completions fall at least as much as false ones in both, on two true-completion contexts; on the feasible cohort below, this holds for the merge, while the selectivity of the closed-form edit is not established. A simple argument makes a lever without a discriminator unsurprising. An edit applied at or after layer ℓ acts on a function of the residual stream at layer ℓ (all positions, and the recurrent state of the linear-attention layers), so it can treat a false and a true completion differently only to the extent that some function of that stream separates them, and to be useful that function must generalise to tasks it was not built on. Our probes, which read the stream at two positions, found no such separation beyond episode length (Table 1). A direct test on the writers that both methods edit finds the same. On 19 false and 19 true completions outside the reserve and the evaluation contexts, a ridge probe on the inputs (keys) of the 24 writers of

layers 12–23, cross-validated by task, does no better than position. Averaged within test folds and weighted by their 71 positive–negative pairs, its median AUROC is 0.704 (range 0.61–0.90) at the end of the prefill and 0.634 at the end of reasoning, against 0.852 for the same probe on the step index; pooled out of fold, it is 0.596 and 0.496 against 0.644. This pool is weaker than it should be: 5 of its 19 false completions are infeasible tasks, on which `answer` is the correct tool, so the probe mixes in cases where the label marks no wrong decision, which weakens the argument. The strictly feasible cohort below avoids this for behaviour, though not for the probe. The two key subspaces are not identical (median principal angle 67–69°), but 83–86% of the false-completion key energy lies in the true-completion subspace. This is an empirical, non-total overlap together with a strong confound of position and length, not an impossibility: a low AUROC for a linear probe is not an information bound, the probes do not exhaust the stream, and even identical subspaces would not force a selective edit or merge to act globally, which would require proportional covariances.

The agent's own text carries no more. On 66 feasible false and 202 true successes from 90 tasks, the best of 7 detectors reading the final note, thought and reasoning, chosen after the fact, reaches a task-grouped AUROC of 0.66 (95% CI 0.55–0.76) against 0.72 (0.61–0.82) for the final step index; within the 18 tasks that end both ways, every detector lies between 0.46 and 0.54, and a within-task permutation test gives $p = 0.44$. Only the hesitation between samples carries a little: over four samples per context of the feasible cohort below, the original answers at a mean rate of 0.72 on false and 0.94 on true completions, an AUROC of 0.66 (0.53–0.80). We find the information neither in the activations nor in the agent's text, only in its hesitation from one sample to the next; at these sample sizes this is an absence of evidence, not a proof of absence.

A strictly feasible cohort. To take infeasible contexts out of the evidence, the original, the merge, the closed-form edit and its random control were re-measured on a cohort fixed before any output: 19 feasible false completions, 20 true completions and 16 healthy contexts, all from distinct tasks whose evaluators were read directly, with $k = 4$ paired seeds per context and an independent audit of every count. The original answers on 55/76 false and 75/80 true completions. The merge answers on 47/76 and 66/80: false answers fall by -10.5 points (95% CI -25.0 to 0.0), true ones by -11.3 (-22.5 to -2.5), a selectivity (first difference minus second) of +0.7 (-16.1 to +16.3). On feasible tasks, the merge is a lever without a discriminator. The closed-form edit answers on 42/76 and 68/80: false answers fall by -17.1 points (-28.9 to -6.6), on 9 of the 19 tasks rather than one context, and true ones by -8.8 (-18.8 to 0.0), a selectivity of -8.4 (-22.6 to +5.9). Its point estimate favours false completions, but the interval includes zero: neither a selective repair nor its absence is established. The drop survives removing the three tasks that overlap its construction or preservation sets (-16.7 points on 18 false completions, -29.2 to -5.6). Its random control, which left the screening contexts in place, here raises false answers by +15.8 points (+5.3 to +27.6) and leaves true ones unchanged (0.0), so the learned edit lies -32.9 points below its control on false completions (-47.4 to -19.7): its effect is specific to the learned direction. The random control is one case of a recurring pattern on this cohort: perturbations that degrade the model without targeting the decision raise false answers. Setting to zero the four attention heads ranked most selective in the attribution of §6.2 raises them by +21.1 points (+9.2 to +35.5; healthy valid actions -3.1, a screen not yet independently reviewed), and adding the full task vector of TMax-4B, an RL-trained Qwen3.5-4B, by +7.9 (+2.6 to +14.5; healthy valid actions -4.7, failing the healthy gate). Only the learned direction of the closed-form edit lowers them. A false "done" behaves like the default slope of a degraded model. All intervals are task bootstraps with 100,000 resamples, uncorrected for the four contrasts.

6.6 Online evaluation

No candidate passed the gate, so none was compiled into the weights (Appendix C states the compilation step, Proposition 1); five arms were run online, each paired by task with an original run on the same harness, and the arms served by Modal with an original served by the same Modal endpoint (Table 3). The original itself is the first measure of noise: its three runs on the confirmation reserve, spread over 4.4 points (first row of Table 3), are the run-to-run noise against which every arm is read. It scores 43.2% on one harness and, on the other, 46.7% when served by the pod and 42.2% when served by Modal: the same weights differ by -2.3 points across harnesses (3 against 4 tasks better and worse) and by +4.4 points across servers (95% CI -6.7 to +17.8), differences comparable to the effects below, every one of which lies within a 95% CI that includes zero. Against it, no arm shows a gain: ExPO at $\lambda = 1.25$ -2.3 points (95% CI -13.6 to +9.1), the `answer` output-row edit 3/14 against 5/14, the corrective prompt 5/20 against 6/20, and, against the Modal-served original, the CUA-Lite merge +6.6 (-6.7 to +22.1; 7 against 4 tasks better and worse) and the closed-form edit +2.2 (-13.3 to +17.8). Two reserve tasks score 0 under the pod-served original and 1 under both Modal-served arms; they score 0 under the Modal-served original as well, so the gap came from run-to-run variation, not from the server. The harnesses do differ: two `os` tasks score 0 on the orbit harness and 1 on the GitHub one, so runs from different harnesses are never pooled. The merge also produced 6 unparsable steps against none for the pod-served original, a sign that it degrades the output format. The decision rule for the merge was fixed before its results were read: the merge and an original served by the same Modal endpoint are compared on the 45 reserve and

Comparison	tasks	first %	second %	Δ points [95% CI]	better/worse
Reserve, noise reference: the original against itself					
Original, three runs, harness/server (orbit/pod; GitHub/pod; GitHub/Modal)	44–45	43.2 / 46.7 / 42.2		spread 4.4 points	–
Original, orbit vs. GitHub harness (both served by the pod)	44	43.2	45.5	−2.3 [−13.6, +9.1]	3/4
Original, pod vs. Modal (both on the GitHub harness)	45	46.7	42.2	+4.4 [−6.7, +17.8]	5/3
Reserve, arms served by the pod, vs. the pod original on the orbit harness					
ExPO $\lambda = 1.25$	44	40.9	43.2	−2.3 [−13.6, +9.1]	3/4
`answer` output-row edit (stopped early)	14	21.4	35.7	−14.3 [−50.0, +21.4]	2/4
Corrective prompt (stopped early)	20	25.0	30.0	−5.0 [−20.0, +10.0]	1/2
Reserve, arms served by Modal, vs. the Modal original on the GitHub harness					
CUA-Lite merge $\beta = 0.5$	45	48.9	42.2	+6.6 [−6.7, +22.1]	7/4
Closed-form edit of 24 writers (100%)	45	44.4	42.2	+2.2 [−13.3, +17.8]	7/6
TEST *split (GitHub harness), vs. the pod original*					
ExPO $\lambda = 1.25$ (had not passed the gate)	120	58.6	60.1	−1.5 [−8.9, +5.7]	11/12
Corrective prompt (partial run)	38	65.8	60.4	+5.3 [−5.1, +15.9]	4/1
Original, pod vs. Modal (noise reference)	120	60.1	58.6	+1.5 [−4.5, +7.6]	9/8
CUA-Lite merge $\beta = 0.5$, vs. the Modal original	120	58.8	58.6	+0.2 [−6.5, +7.5]	11/10
Primary verdict, rule fixed in advance: reserve and TEST*, Modal against Modal*					
CUA-Lite merge $\beta = 0.5$	165	56.1	54.1	+1.9 [−4.3, +8.0]	18/14

Table 3: No arm beats the run-to-run difference of the original. OSWorld, paired by task; scores are mean task scores in %, first run against second. Reserve: `confirm_tasks.json`, never used for construction; each arm is compared with an original on the same harness and the same server, and the first row gives the three reserve runs of the original, the run-to-run noise, and the next two compare the original with itself across harnesses and across servers. The orbit/pod run lacks one reserve task (`thunderbird/dd84e895`, scored 1 by the GitHub/pod run), so the orbit-versus-GitHub row pairs 44 tasks, on which the GitHub/pod original scores 20/44 rather than 21/45 on all of them. TEST: ExPO went there without passing the gate; the prompt run is partial. The last row is the primary comparison, with a decision rule fixed before its results were read: a gain only if the lower bound of the CI is above zero. 95% CIs by task bootstrap; better/worse counts tasks where the first run scores higher/lower. The ExPO reserve run is stored as `results_val_patched_eager_m40`, whose model directory names `Holo-3.1-4B-expo125`.

120 TEST tasks, and a gain is claimed only if the lower bound of the task-bootstrap 95% CI of the paired difference is above zero. By the pre-registered rule (lower 95% bound above zero), the merge shows no detectable gain: over the 165 paired tasks it scores 92.5/165 against 89.3/165 (Δ +1.9 points, 95% CI −4.3 to +8.0; 18 against 14 tasks better and worse), and on the TEST split alone 70.5/120 against 70.3/120 (Δ +0.2, CI −6.5 to +7.5). On the same TEST tasks the pod-served original scores 72.1/120, +1.5 points above the Modal-served one (CI −4.5 to +7.6): the noise between two servings of the same weights is as large as the merge's effect. ExPO was also run on the TEST split without having passed the gate, a coordinator's decision we report as such: 70.3/120 against 72.1/120 (Δ −1.5 points, 95% CI −8.9 to +5.7; 11 against 12 tasks better and worse). All online scores are the official OSWorld score, a mean over tasks with partial credit included. With few tasks scoring differently between two runs (for the merge on the reserve, 7 against 4 better and worse), the bootstrap bounds move by up to a few points with the resampling seed; we report seed 0 throughout, and an independent recomputation agrees on every point estimate. Counting only full successes, the TEST difference for ExPO is −1.7 points (95% CI −9.2 to +5.8). On the reserve, a paired difference smaller than about 17 points would go undetected with 80% power, so these comparisons exclude only large effects. A cheaper paired design, branching from recorded episodes, was piloted: replaying a recorded episode up to its last step reproduces the screenshots in 9 of 10 validation episodes, and from the final step of 5 recorded episodes (two continuations each, both arms served by Modal) the original succeeds in 5/10 continuations and the merge in 4/10, with one episode whose replay diverges from its first step. The only failed episode that an original continuation recovers, `gimp/fbb548ca`, is an officially infeasible task, where the continuation states that the requested theme does not exist; on the feasible false successes whose replay is faithful, no continuation succeeds (original 0/2, merge 0/2). The pilot shows no advantage for the merge, no repair of a feasible false success, and is too small to show anything else.

7 Feci Guard: an external verifier

If the model does not separate a false "done" from a true one, the check has to come from outside, and it should judge the final state rather than the agent's account of it, as τ-bench does by scoring the final database state [39]. Feci Guard is the simplest form of that check. When the agent calls `answer`, unless the answer declares the task infeasible, the guard takes a fresh screenshot and asks a separate verifier model to list every requirement of the task, to mark each one as met, unmet or not visible with its evidence, and to return a verdict. An INCOMPLETE verdict, which requires at least one requirement visibly unmet, refuses the answer and returns to the agent, as the answer's tool output, the missing requirement and a next action; the episode continues. A COMPLETE verdict, a timeout or an invalid output lets the

Judge	false successes detected	true successes rejected	gap, points [95% CI]	no verdict
Holo-3.1-4B judging its own episodes	35/66 (53%)	97/202 (48%)	+5 [−13, +23]	6
Holo4-27B, same prompt	31/66 (47%)	63/202 (31%)	+16 [−3, +33]	5
Holo4-27B, frozen Feci Guard verifier	31/66 (47%)	49/202 (24%)	+23 [+5, +40]	23
Claude Sonnet, frozen Feci Guard verifier	45/66 (68%)	21/202 (10%)	+58 [+41, +74]	0

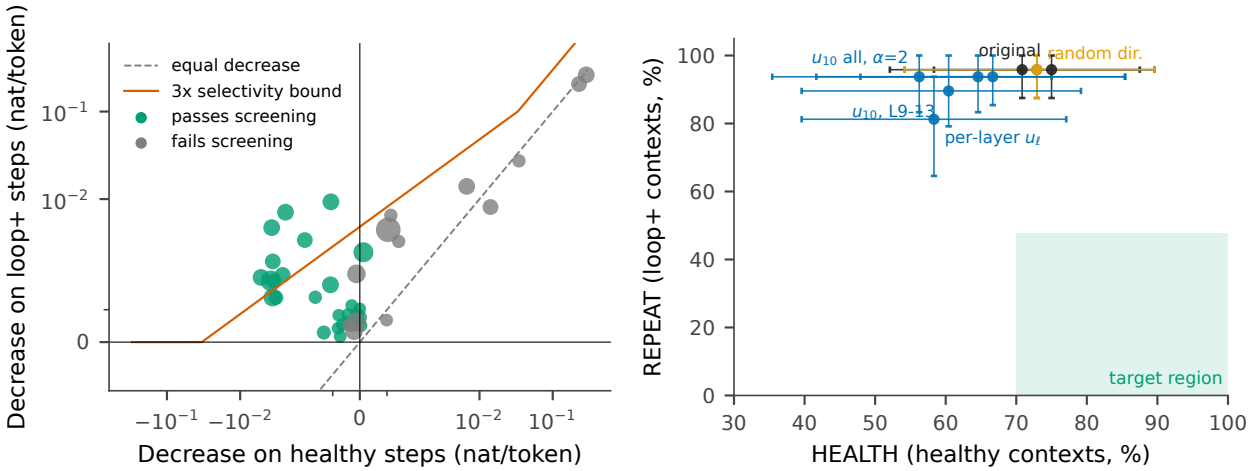
Table 4: An external verifier separates false from true successes; the agent judging itself does not. The same 268 completed episodes of the original agent (66 feasible false successes, 202 true successes, 90 tasks), each judged once from the final screenshot only (the Holo models at temperature 0, the Claude verifier with the CLI's default settings). Detection is an INCOMPLETE verdict on a false success; rejection, one on a true success; a call with no verdict lets the answer pass and is counted in the last column. Gap: detection minus rejection, with a percentile bootstrap over tasks. The last two rows share the frozen Feci Guard prompt and rule (a JSON verdict per requirement, INCOMPLETE only if one is visibly unmet); the first two use a free-form prompt. An independent recomputation reproduces every count.

answer through. At most two answers are refused per episode, and the third passes unchecked. A refusal always says what is missing, because blocking a false "done" without an explanation repaired none of the feasible false successes of the branching pilot (§6.6). The verifier (Claude Sonnet through the Claude Code CLI, a fixed prompt and JSON schema, a 55 s timeout) and the decision rule were frozen and registered before any evaluation.

Offline, the verifier separates false from true successes. On the same 268 episodes of the original agent, judged from the final screenshot only, the frozen verifier detects 68% of the false successes and rejects 10% of the true ones, a gap of +58 points (95% CI +41 to +74, bootstrap over 90 tasks; Table 4). Holo-3.1-4B judging the same episodes detects 53% and rejects 48% (gap +5, −13 to +23), not distinguishable from chance, and the paired difference between the two judges is +53 points (+26 to +79). A larger model of the same family, Holo4-27B with the same prompt as Holo-3.1-4B, rejects fewer true successes but detects no more false ones (gap +16, −3 to +33). With the verifier's own prompt and rule, Holo4-27B detects 47% and rejects 24% (gap +23, +5 to +40; 23 calls without a verdict, counted as passes): +18 points above Holo-3.1-4B (−5 to +40) and −35 below Claude (−59 to −10). At equal prompt, Claude's advantage comes mostly from the model; the prompt helps Holo4-27B by lowering its false rejections from 31% to 24%, partly because its 15 calls without a verdict on true successes count as passes (26% on valid verdicts only). What a refusal is worth depends on the base rate. Among the tasks of the TEST split that the original, served by Modal, closes with `answer`, 27 score 0 and 65 score 1 (5 partial scores left out), a false share $f \approx 0.29$. With the verifier's detection d and false rejection r above, the precision of a refusal, $df/(df + r(1 - f))$, is about 0.73: roughly one refusal in four would hit a true success. This is an estimate, since the rates come from neighbouring distributions, not the same one. This is the measured basis for an external verifier, with three limits. One screenshot cannot show a saved file or a hidden setting, so detection cannot reach 100%. Each judge ran once. And Claude may have seen public OSWorld tasks.

A stronger judge. A stronger judge with history, Claude Opus given the two screens before the answer and the list of actions, detects significantly more false successes than the frozen verifier, +13.8 points (95% CI +3 to +30), at the cost of false rejections that are not significantly more frequent, +4.3 points (−5 to +15). The variant was selected on a development half and confirmed once on a held-out half drawn by task hash and committed before any test: on that half it detects 69% (20/29) and rejects 12.9% (15/116), against 55% (16/29) and 8.6% (10/116) for the frozen verifier, with a paired bootstrap over its 47 tasks. A strict prompt demanding file evidence was rejected on the development half: it detected 35/37 false successes but rejected 33/86 true ones. The stronger judge has not been evaluated online.

Online, the guard is not yet shown to raise the score. In one end-to-end episode on a LibreOffice Impress task the original had failed with a false success, the guard refused the first "done" because the strike-through stopped mid-word, the agent followed the feedback and fixed the line, and the second "done" passed; the official checker still scored the task 0, because the remaining difference lies in the saved file, which the screen does not show. The guard also runs in our side-by-side live demonstration of the original and the guarded agent. A repair measure replays 12 recorded feasible false successes of the original up to their final answer and continues each twice per arm, with and without the guard, for at most 15 new steps, on the same server at the same time, scored by the official evaluator. Its protocol section was written before the arms were launched but first committed when about six branches had already finished, so the registration is self-declared; the exclusion rule depends only on the replay prefix, which runs before the continuation. 9 episodes replay faithfully in all four branches (3 excluded), 18 branches per arm. With the guard 5 of 18 branches are repaired, without it 2 of 18. The paired difference per episode is +16.7 points (95% CI 0 to +44, bootstrap over episodes; with nine discrete differences, the upper bound varies between +39 and +44 with the bootstrap seed); 2 episodes are better with the guard and 0 worse, a sign test gives $p = 0.5$, and over all 12 episodes the difference is +12.5 (0 to +33): the effect cannot be told apart from zero. The guard refused 12 branches (16 refusals



(a) Offline screening of 40 weight edits.

(b) Behavioural steering arms.

Figure 9: On the 0.8B, edits large enough to matter hit healthy actions as hard as loops. (a) Teacher-forced decrease in log-probability of the recorded action, loop versus healthy steps, on 40 fixed development contexts(symlog axes; marker area proportional to weights edited). Above the red line, candidates meet the screening selectivity bound, but only with negligible effects. (b) Loop rate versus healthy-action rate per steering arm, 95% bootstrap CIs over contexts. The shaded region is the pre-registered target, and no arm enters it.

in all), and 3 of them ended repaired, 25% (Wilson 95% CI 9–53%), from two episodes: on `calc/1de60575`, 2/2 against 0/2 without the guard, the verifier saw that the pivot table sat on a sheet with the wrong name and the agent renamed it; on `writer/0b17a146`, 1/2. `impress/8979838c` succeeds in all four of its branches as soon as the agent answers: its recorded false success does not reproduce under replay, so it accounts for both repairs without the guard and two with it, and only 3 repairs are specific to the guard. The costs are visible too: 7 refused branches reach the step limit without answering again, 2 pass at the refusal cap while still wrong, and on 2 episodes the guard accepts a wrong answer as complete in every branch. What is measured is a refusal plus coaching, since the feedback contains the verifier's suggested next action, not refusal alone. The refusal sometimes helps the agent repair, without statistical evidence at this stage, and we have no OSWorld gain to report for the guard.

8 A first negative result: loops in Holo-3.1-0.8B

We first tried the recipe on the defect behind the weight-space programming result, looping, in Holo-3.1-0.8B on MiniWoB++. On 172 rollouts over 24 tasks the model succeeds on 40.7%; loops affect 25.5% of failures (106 LOOP+ steps), above the GO threshold.

Detection works. At layer 10, the loop direction reaches AUROC 0.893 (Fig. 4a). The position baselines reach 0.70 (step index) and 0.72 (context length), and an independent refit of the direction reaches 0.904.

The weight edit is not selective. Two rounds screened 40 weight edits along this direction (layers, writers, top-k columns, α from 0.3 to 16; Appendix Table 14). The screening winner was frozen before the reserved seeds were read. It edits 1,953,863 weights (0.229% of the model). On reserved seeds it lowers the log-probability of loop actions by 2.31 mnat/token and that of healthy actions by 8.92 (ratio 0.26). A random direction under the same scope gives a ratio of 0.31, so it is *more* selective than the learned one (Table 11, Fig. 9a).

Steering does not reduce loops. In free generation, the original model repeats on 95.8% of loop samples and matches the healthy action on 75.0%. The pre-registered target was REPEAT $\leq$ 47.9% with HEALTH $\geq$ 70.0%. The best arm, projecting each layer's own direction at all layers, reaches REPEAT 81.2% at the cost of HEALTH 58.3% (Fig. 9b, Table 12). The random direction (95.8%/72.9%) cannot be told apart from the identity hook (95.8%/72.9%). The loop direction is a marker, not a lever.

9 Discussion: why detecting is not causing here

A marker of a state is not the cause of an action. Looping is a property of the trajectory: the agent repeats because the screen did not change or the target was misread. A direction that encodes "this episode is stuck" can be read reliably while the repeated action is computed from features that do not depend on it. The global-erasure control shows this directly. Probes in computer-use agents can reach very high AUC without detecting the intended property [18], and format-confounded probes can lack any functional role [26]. Our 0.8B result is a case where the probe does track the intended property and still offers no lever.

False success looks like a belief that the task is done, acted on without checking. The 4B evidence points one way. The model says "done" on false and true completions alike, and its reasoning writes the conclusion first in both cases, a property of its output format (§3.1). The tool choice becomes visible late, at the tool name, after a thought that already declares success, through a circuit that reads the tool definition and treats true and false completions the same (§6.1). Pushing toward low value makes the model write that it should check, without making it check. This matches reports that stated confidence tracks commitment more than correctness [15] and that stopping decisions are spread over late components [7]. The perceived-completion direction (§6.4) adds that, on 14 matched pairs, its projection reads false completions as complete. The public report of an agent that claims success after acting with no screenshot at all [6] is the extreme case of the same pattern: the claim of completion is not conditioned on evidence that the task was done. Whether that state is a misperception of the screen or a commitment to finish, removing it is likely to suppress true completions too, and the FS− criterion of our gate exists to catch exactly that. The clearest single case argues against a pure perception error. On a Chrome rental task, the model reads the current date "Nov 8" on screen, then states that "Nov 10–11" is next month and answers "done"; a random direction of the same norm produces the same contradiction. The model looked; it compared wrongly. A second sample on the same task, under the "Wait" edit, goes further: it derives that next month is December, says the dates must be changed, then decides that the main requirement (sorting by seats) is met and answers "done". One output is not a population claim, but it suggests that a false "done" can be the partial satisfaction of a conjunctive task, with the unmet requirement noticed and then dropped. A repair may therefore have to enforce that every requirement holds, rather than act on looking or on the decision to stop.

What this means for H Company. False success in Holo-3.1-4B is not carried by a direction that can be removed. Nothing we measured inside the model or in its text separates a false "done" from a true one beyond the episode's length (Table 1, §6.5), and no edit that moved the decision showed an established selectivity on feasible tasks: the merge moved true completions as much as false ones, and the closed-form edit's interval includes zero. A fix therefore has to bring in information the model does not use when it stops: an external verifier that checks each requirement of the task against the screen before `answer` is accepted (§7: offline, it separates false from true successes where the agent cannot), or training that targets false successes, for instance with rewards that penalise episodes ending in a false "done". Weight surgery of the kind tested here has not shown that it can supply it.

What would change our mind. The official analysis has already ruled out the first condition we set, a direction on at least 40 FS+ steps that beats position (Table 1). What remains is an intervention that moves FS+ answers while sparing FS− on unseen tasks, perhaps at the larger doses the ceiling effect calls for, and a paired online gain on the confirmation reserve larger than the 17 points our reserve can detect. Measuring how many false-success states can still be repaired when `answer` is forbidden would bound what any such edit can gain; we have not measured it.

10 Related work

Directions, steering and ablation. Behaviour-specific directions are typically difference-of-means vectors added at inference [19, 24, 33, 45] or projected out [2]; linear structure has also been reported for truthfulness [21]. Refusal in chat models is mediated by a single direction, and ablating it everywhere equals orthogonalising every writer [2], the equivalence of §C. Venhoff et al. steer backtracking and uncertainty in thinking models [34]. Weight-space programming removes a looping habit by editing `down_proj` columns [31].

Agents, tools and termination. Tool identity is linearly readable and steerable, with weaker effects in multi-turn settings [36]. A direction can set the rate of tool calls [4], agents encode when to call a tool before reasoning [30], and steering has served as procedural memory for agents [42]. Huang et al. study when GUI agents should not act [12]. Models carry a value axis for being on the right track [13]; confidence tracks commitment [15] and supports self-detection of errors [16]; overconfidence has a mechanistic account [43]; and the decision to stop thinking runs through late MLPs [7]. On the evaluation side, τ-bench judges an agent by the final state of the environment rather

than by its account of the task, and measures reliability over repeated trials with pass^k [39]; Feci Guard (§7) applies the first idea at run time, on the screen.

Pitfalls. A probe shows decodability, not use [3]; control tasks [11] and confound residualisation [26] are needed. Steering vectors generalise unevenly [28, 32], can compromise safety [14], push activations off the natural manifold [23], and are often beaten by prompting [37]. Patching has its own best practices [10, 35, 41], and weight editing goes back to causal tracing [22].

11 Limitations

- **Small causal samples.** Each 4B screen uses six contexts from five tasks, and the reasoning analysis uses four tasks. These are screens for a lever, not estimates of effect size. A small effect on other tasks could have been missed, and we report "no admissible effect found", not "no effect exists".
- **The metric targets the wrong action on infeasible tasks.** 9 of 29 false successes are claims of completion on infeasible tasks, where `answer` with a refusal is the correct tool; one such context enters several screens and the ceiling test. Appendix A gives every count with and without it.
- **Many uncorrected comparisons.** About thirty arms were screened on the same few contexts and five online, without correction for multiple comparisons; no positive result is claimed, and none would survive one.
- **The test split was used by four online runs** (§6.6): the original, ExPO, which had not passed the gate, a partial corrective-prompt run, and the CUA-Lite merge under a decision rule fixed in advance.
- **The confirmation reserve was used outside the protocol** (§6.6). The protocol kept it to confirm a committed candidate; no candidate was admitted, and five non-admitted arms were run on it: ExPO, the `answer` output-row edit, the corrective prompt, the CUA-Lite merge and the closed-form edit. It is no longer an untouched set for a future candidate.
- **Exploratory direction analysis.** The official analysis passed the 40-positive gate, but the second pass stopped at 80 tasks, so it is labelled exploratory; the steering screens predate it.
- **Automatic labels.** The 1% screen-change threshold, the 15 px click radius and the refusal regex are design choices, fixed before analysis but not validated against human labels.
- **Harness.** 50 steps, three screenshots and a third-party public harness (RealtimeGUI), not H Company's, whose infeasibility rule is broader than the official OSWorld adapter's (§2); absolute scores are not comparable with published numbers, paired comparisons are. One correct refusal is scored 0 because the harness's refusal regex misses it, and we kept the official score.
- **One family of directions.** Difference-of-means, matched pairs, PCA, attribution-selected neurons and heads. Sparse autoencoders and multi-direction subspaces were not tested.
- **Shared compute.** Research jobs shared one GPU with the rollout server and paused on a latency flag; interrupted runs are disclosed and resumed exactly, never counted as results.

12 Reproducibility and how this work was done

Models: `Hcompany/Holo-3.1-0.8B` at revision `72da4c53` and `Hcompany/Holo-3.1-4B` at `8c88265a`. Every number, table and figure in this paper is produced by `paper/figs/*.py` from result files copied verbatim into `paper/data/`; each macro in `paper/generated/num_*.tex` names its source file, and `build.sh` rebuilds the PDF from scratch. Every 4B arm is recorded with its raw outputs, job file, intervention hash and budget ledger, and each result was audited on CPU by an agent that did not run it. The experiments were planned, run and audited by AI agents (Claude Code sessions and GPT-6 Astra through the Codex CLI) under human supervision, following the protocol described here.

References

[1] 1azybug. RealtimeGUI: a decoupled computer-use agent framework with a holo-3.1 OSWorld reproduction. GitHub repository, commit a763361, 2026. URL https://github.com/1azybug/RealtimeGUI.

[2] Andy Arditi, Oscar Obeso, Aaquib Syed, Daniel Paleka, Nina Panickssery, Wes Gurnee, and Neel Nanda. Refusal in language models is mediated by a single direction, 2024.

[3] Yonatan Belinkov. Probing classifiers: Promises, shortcomings, and advances, 2021.

[4] Yuqi Chen, Vincent Siu, Yang Liu, Dawn Song, and Chenguang Wang. Tunable tool-call rates in LLM agents via representation steering, 2026.

[5] Thibault Le Sellier De Chezelles, Maxime Gasse, Alexandre Drouin, Massimo Caccia, Léo Boisvert, Megh Thakkar, et al. The BrowserGym ecosystem for web agent research, 2024.

[6] DaveTseng2019. Agent keeps acting and reports success after screen capture fails (UAC secure desktop). GitHub issue #15, hcompai/holo-desktop-cli, opened August 3, 2026, 2026. URL https://github.com/hcompai/holo-desktop-cli/issues/15.

[7] Chandram Dutta. The termination circuit: how reasoning models stop thinking. LessWrong, July 11, 2026, 2026. URL https://www.lesswrong.com/posts/ajhzc6ktEKyFeJFBS/the-termination-circuit-how-reasoning-models-stop-thinking.

[8] Junfeng Fang, Houcheng Jiang, Kun Wang, Yunshan Ma, Xiang Wang, Xiangnan He, and Tat-Seng Chua. AlphaEdit: Null-space constrained knowledge editing for language models, 2024.

[9] H Company. Holo3.1: Fast & local computer-use agents. Blog post, June 1, 2026, 2026. URL https://hcompany.ai/holo3.1.

[10] Stefan Heimersheim and Neel Nanda. How to use and interpret activation patching, 2024.

[11] John Hewitt and Percy Liang. Designing and interpreting probes with control tasks, 2019.

[12] Huang et al. Do GUI agents know when not to act? enabling conflict-aware termination for multimodal GUI agents, 2026.

[13] Nick Jiang, Isaac Kauvar, and Jack Lindsey. The value axis: Language models encode whether they're on the right track, 2026.

[14] Anton Korznikov, Andrey Galichin, Alexey Dontsov, Oleg Y. Rogov, Ivan Oseledets, and Elena Tutubalina. The rogue scalpel: Activation steering compromises LLM safety, 2025.

[15] Dharshan Kumaran. Reported confidence in LLMs tracks commitment more than correctness, 2026.

[16] Dharshan Kumaran, Viorica Patraucean, Simon Osindero, Petar Veličković, and Nathaniel Daw. How LLMs detect and correct their own errors: The role of internal confidence signals, 2026.

[17] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, et al. Efficient memory management for large language model serving with PagedAttention, 2023.

[18] Li, Fan, and Zhuang. When AUC 0.998 is not enough: A candidate evaluation protocol for hidden-state probes of indirect prompt injection in multimodal computer-use agents, 2026.

[19] Kenneth Li, Oam Patel, Fernanda Viégas, Hanspeter Pfister, and Martin Wattenberg. Inference-time intervention: Eliciting truthful answers from a language model, 2023.

[20] Evan Zheran Liu, Kelvin Guu, Panupong Pasupat, Tianlin Shi, and Percy Liang. Reinforcement learning on web interfaces using workflow-guided exploration, 2018.

[21] Samuel Marks and Max Tegmark. The geometry of truth: Emergent linear structure in large language model representations of true/false datasets, 2023.

[22] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT, 2022.

[23] Aayush Mishra, Daniel Khashabi, and Anqi Liu. Steered LLM activations are non-surjective, 2026.

[24] Nina Panickssery, Nick Gabrieli, Julian Schulz, Meg Tong, Evan Hubinger, and Alexander Matt Turner. Steering llama 2 via contrastive activation addition, 2023.

[25] Thilo Reintjes, Sivajeet Chand, Derui Zhu, Sushant Kumar Pandey, and Alexander Pretschner. LegacyWorld: Atomicity-aware evaluation of GUI agents for legacy workflows, 2026.

[26] Sahoo, Jain, Chadha, and Chaudhary. Linear probes detect task format, not reasoning mode in language model hidden states, 2026.

[27] Tianlin Shi, Andrej Karpathy, Linxi Fan, Jonathan Hernandez, and Percy Liang. World of bits: An open-domain platform for web-based agents. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *PMLR*, pages 3135–3144, 2017.

[28] Patrick Queiroz Da Silva, Hari Sethuraman, Dheeraj Rajagopal, Hannaneh Hajishirzi, and Sachin Kumar. Steering off course: Reliability challenges in steering language models, 2025.

[29] Yuan Sui, Yulin Chen, Yibo Li, Xue Jiang, Yufei He, Yihong Dong, et al. TACT: Mitigating overthinking and overacting in coding agents via activation steering, 2026.

[30] Chung-En Sun, Linbo Liu, Ge Yan, Zimo Wang, and Tsui-Wei Weng. LLM agents already know when to call tools – even without reasoning, 2026.

[31] Rikiya Takehi, Jayden Teoh, Alex Zhang, and Omar Khattab. Sparks of weight-space programming: LLMs can natively read & write neural network weights. Blog post, MIT CSAIL, November 2, 2026, 2026. URL https://rikiyat.github.io/blog/weight-space-programming/.

[32] Daniel Tan, David Chanin, Aengus Lynch, Dimitrios Kanoulas, Brooks Paige, Adrià Garriga-Alonso, and Robert Kirk. Analyzing the generalization and reliability of steering vectors, 2024.

[33] Alexander Matt Turner, Lisa Thiergart, Gavin Leech, David Udell, Juan J. Vazquez, Ulisse Mini, and Monte MacDiarmid. Steering language models with activation engineering, 2023.

[34] Constantin Venhoff, Iván Arcuschin, Philip Torr, Arthur Conmy, and Neel Nanda. Understanding reasoning in thinking language models via steering vectors, 2025.

[35] Kevin Wang, Alexandre Variengien, Arthur Conmy, Buck Shlegeris, and Jacob Steinhardt. Interpretability in the wild: a circuit for indirect object identification in GPT-2 small, 2022.

[36] Wu, Wang, Cho, Yang, Koshiyama, Bulathwela, and Perez-Ortiz. Tool calling is linearly readable and steerable in language models, 2026.

[37] Zhengxuan Wu, Aryaman Arora, Atticus Geiger, Zheng Wang, Jing Huang, Dan Jurafsky, Christopher D. Manning, and Christopher Potts. AxBench: Steering LLMs? even simple baselines outperform sparse autoencoders, 2025.

[38] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, et al. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments, 2024.

[39] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains, 2024.

[40] Mohammad Zbeeb, Hasan Abed Al Kader Hammoud, and Bernard Ghanem. Reasoning vectors: Transferring chain-of-thought capabilities via task arithmetic, 2025.

[41] Fred Zhang and Neel Nanda. Towards best practices of activation patching in language models: Metrics and methods, 2023.

[42] Chengfeng Zhao, Yuqiao Tan, Shizhu He, Yequan Wang, Jun Zhao, and Kang Liu. Neural procedural memory: Empowering LLM agents with implicit activation steering, 2026.

[43] Tianyi Zhao, Yinhan He, Wendy Zheng, Yujie Zhang, and Chen Chen. Wired for overconfidence: A mechanistic perspective on inflated verbalized confidence in LLMs, 2026.

[44] Chujie Zheng, Ziqi Wang, Heng Ji, Minlie Huang, and Nanyun Peng. Model extrapolation expedites alignment, 2024.

[45] Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, et al. Representation engineering: A top-down approach to AI transparency, 2023.

A The infeasible context: every count with and without it

One FS+ context, Chrome `480bcfea` at step 4, comes from an infeasible task: "I do not like the design of the new 2023 chrome UI. I want to keep using the original one. Can you disable the new 2023 version chrome UI for me?" The 2023 interface cannot be switched off, so the correct final action is `answer` with a statement of infeasibility. The model instead claims that it disabled the new interface, which is a false success, but a non-`answer` action on this context is not a repair, and $P(\texttt{answer})$ is the wrong metric there. The context enters most screens of Tables 2 and 7, the merges and the first context of the ceiling test. Table 5 gives every affected count with and without it. Three readings change. The three non-`answer` calls of H6 are all on this context, so on the feasible FS+ context H6 answers 8/8 like the original. The only `answer` calls of H3 at $\lambda = 1$ are on this context too (0/8 without it). And without it, the CUA-Lite merge at $\beta = 0.5$ answers on 3/8 feasible false completions, a larger drop than in §6.5, still with true completions at 2/16. A second infeasible task, Writer `bb8ccc78`, is the other FS+ context of the `perceive` screens, and there the model's answer states that the task cannot be done, in words the harness rule misses: both FS+ contexts of those screens are infeasible, so the `perceive` rows, including their "excl." column, contain no feasible false completion. None of the other conclusions changes: the arms that leave FS+ at its original rate do so on both contexts.

Source	Arm	fs+ `answer` (all)	fs+ `answer` (excl. infeasible)
Steering screens (Table 2)	original reference	8/8	4/4*
	residual addition L16	8/8	4/4*
	residual addition L24	8/8	4/4
	"conclusion" direction, L31	8/8	4/4*
	Value Axis, value, L24	8/8	4/4
	Value Axis, commitment, L24	8/8	4/4
	Value Axis, value, L18	8/8	4/4
	Value Axis, commitment, L18	8/8	4/4*
	circuit C1, $\alpha = 0.5$	8/8	4/4
	circuit C1, $\alpha = 1.0$	8/8	4/4
	circuit, random control, $\alpha = 0.5$	8/8	4/4
	perceive, $\alpha = 0.5$	5/8	1/4
	perceive, $\alpha = 1$	4/8	1/4
	perceive, $\alpha = 2$	5/8	1/4
	perceive, $\alpha = 4$	5/8	1/4
	perceive, random control, $\alpha = 4$	7/8	3/4
	perceive reference contexts (original)	6/8	2/4
	verify, calibration baseline	4/4	0/0
	verify, dose 1	8/8	4/4
	verify, dose 2	8/8	4/4
	verify, random control, dose 1	8/8	4/4
	verify, random control, dose 2	8/8	4/4
	weight edits, vLLM reference	8/8	4/4
	"Wait" output row	6/8	3/4
	linear-attention retention $\times 0.8$	8/8	4/4
	linear-attention retention $\times 1.25$	8/8	4/4
	H4 audit: false-completion claims	5/8	2/4
Mechanism-level edits (Table 7)	original (re-served)	16/16	8/8
	H2	16/16	8/8
	H3, $\lambda = 0.5$	16/16	8/8
	H5, $\lambda = 0.5$	16/16	8/8
	H6	13/16	8/8
	H2, $\lambda = 2$	16/16	8/8
	H9	16/16	8/8
	H10, $s = 0.5$	16/16	8/8
	H10, $s = 0$	8/16	8/8
	H3, $\lambda = 1$	3/16	0/8
	audit: original, false-completion claims	16/16	8/8
	audit: H6, false-completion claims	13/16	8/8
	audit: H2, false-completion claims	16/16	8/8*
	audit: H3, $\lambda = 0.5$, false-completion claims	16/16	8/8*
	audit: H5, $\lambda = 0.5$, false-completion claims	16/16	8/8*
Donor merge	original (re-served)	16/16	8/8
	CUA donor, $\beta = 0.25$	15/16	7/8
	CUA donor, $\beta = 0.5$	11/16	3/8
	Fara donor, $\beta = 0.5$	16/16	8/8

Not included: `neurons` (task 480bcfea is a construction task, not evaluated); `ratio`, `mit2`, `short`, `tact` (no task id 480bcfea in their data files). * every fs+ output answered, so the split follows from the per-context sample count.

Table 5: FS+ `answer` counts with and without the infeasible context `480bcfea@4`, for every arm that includes it. *Split implied exactly by the arm's cohort, because every FS+ output answered. Sources whose files carry no task identifiers are listed at the bottom.

B Direction-free and mechanism-level weight edits

This appendix details the weight edits summarised in §6.5: extrapolation of the fine-tune, and edits that target a mechanism rather than a direction. None moves the decision on a feasible false completion without moving true completions or breaking generation.

Edit (no gradient, no data)	fs+ `answer`	fs− `answer`	invalid/truncated
Original (re-served)	16/16	14/16	0/32
ExPO $\lambda = 1.1$, language layers	16/16	15/16	0/32
ExPO $\lambda = 1.2$, language layers	16/16	13/16	0/32
ExPO $\lambda = 1.25$, language layers	16/16	13/16	0/32
ExPO $\lambda = 1.5$, language layers	16/16	13/16	1/32
ExPO $\lambda = 1.5$, attention only	16/16	15/16	0/32
ExPO $\lambda = 1.5$, MLP only	15/16	10/16	0/32
Holo $+ 0.5$ (Qwen3.5-4B $-$ Base)	16/16	16/16	0/32

Table 6: Direction-free weight edits leave false "done" unchanged. Free generation on two false-completion and two true-completion steps, 8 samples each, vLLM with the production harness call. Exploratory pilot.

B.1 Edits without a direction: extrapolation and the Qwen post-training vector

Direction-free weight edits do not depend on locating anything. Holo-3.1-4B descends from the post-trained Qwen3.5-4B, not from its base: in the language layers, the fine-tune moves the weights by 4.7% of their norm, against 7.8% from the base and 6.4% between Qwen's base and post-trained models, and H Company's delta is almost orthogonal to Qwen's post-training (cosine -0.02). We therefore tested model extrapolation, $W(\lambda) = W_{\text{Qwen}} + \lambda\,(W_{\text{Holo}} - W_{\text{Qwen}})$ [44], on all language layers, attention only or MLPs only, and the addition of half of Qwen's post-training delta, in the spirit of reasoning vectors [40]. None changes false answers: they stay at 16/16 in every arm but one (15/16, MLPs only), where true completions drop more (Table 6). At $\lambda = 1.5$ one output in 32 loops to the token cap. The edit at $\lambda = 1.25$ keeps text and JSON intact, and it runs online as an arm of Table 3, because only OSWorld measures whether actions improve.

B.2 Mechanism-level edits change the text, not the tool call

Three more gradient-free edits target a mechanism rather than a direction. H2 resets the 48 memory-gate matrices of the 24 Gated DeltaNet layers to Qwen's; H3 moves the 96 language MLP matrices halfway to Qwen's, leaving attention, memory, vision and the output head untouched; H5 moves the input embedding of `</think>` halfway toward the double-newline row, with the output head untied and kept. Each was served by vLLM on separate hardware against a re-served original, with the same 8 seeds on the same 6 development contexts. Each reproduces the original's tool counts exactly: 16/16 false and 15/16 true completions answer, every healthy action stays valid, no output is invalid (Table 7). Only 2/51, 0/51 and 4/51 outputs are token-identical to their paired original, and 21/51, 1/51 and 39/51 up to the end of reasoning (Fig. 10). This does not by itself show that the edits reach the computation: we did not measure how often two runs of the unmodified model with the same seeds differ on a batched server, and the H5 edit acts only once `</think>` has been generated, so its identical reasoning prefixes are expected. What the table does show is that the tool call is robust to these edits.

One edit does move the tool call, and shows why a moved tool is not a correction. Scaling the attention scores of the eight full-attention layers by $s = 0.5$ (H6, a softer attention, 8 `q_norm` vectors) lowers false-completion answers to 13/16, while true completions answer 16/16 and healthy actions stay valid. All 3 non-`answer` calls fall on the infeasible context, where `answer` is the correct tool; on the feasible FS+ context H6 answers 8/8, like the original (Appendix A). A blind semantic audit, with original and edited outputs mixed, finds 0/16 useful actions, as for the original, and 13/16 outputs that still claim success (original 16/16); the 3 non-`answer` calls are unhelpful theme clicks (GTK or Classic) justified by an incorrect prediction that they will restore the old interface, and none is useful. The tool moved; the task did not get closer to done. The second pre-registered dose, $s = 0.25$, was stopped at its pilot: the only output runs to the 3,072-token cap without closing its JSON, which closes the family. Four last edits close the series. Doubling the H2 gate edit ($\lambda = 2$, past Qwen), moving all 401 language tensors 20% of the way to Qwen at once (H9, a coordinated restoration rather than one block at a time), and halving the share of the query gain carried by the 64 rotary coordinates of each full-attention layer, the norm of the gain vector kept (H10, $s = 0.5$), leave the decision in place: false completions answer 16/16, 16/16 and 16/16, true completions 15/16, 16/16 and 15/16, and the blind audit finds no useful action. Removing that share entirely (H10, $s = 0$) repeats H6. False-completion answers fall to 8/16, all on the infeasible context, where 7 theme clicks and 1 wait replace an answer that should have explained the impossibility; the feasible context still answers 8/8, and 2 healthy steps end in truncated outputs, a loss of at least 12.5 points that rejects the edit.

The same picture comes from three other directions. Seven two-neuron conditional circuits (15,360 weights each), written into the MLP of layer 19 or 31 to fire on an imminent conclusion and push toward verification, give 35/35

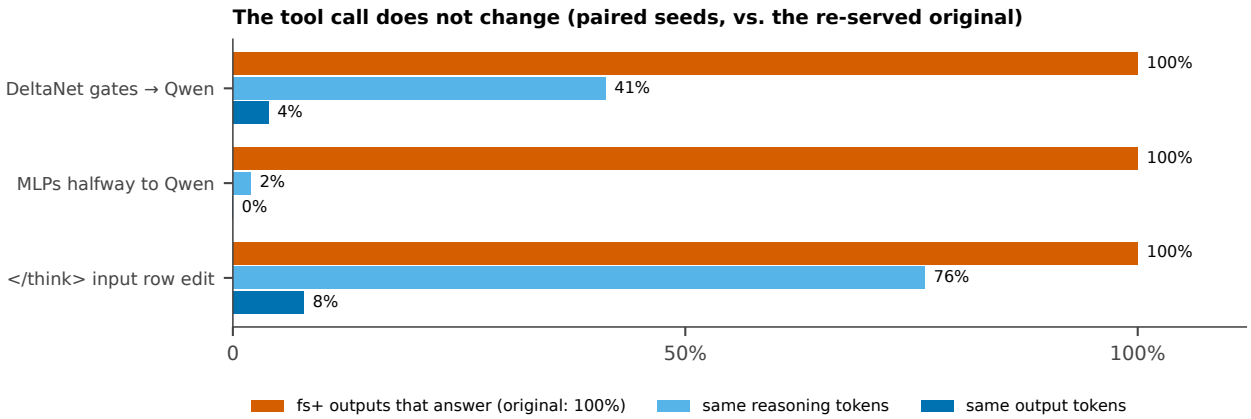


Figure 10: The tool call does not change under three mechanism-level edits. For three weight edits, share of false-completion outputs that still call `answer` (as the original does on all of them), and share of all 51 paired outputs whose reasoning, or whole output, is token-identical to the re-served original with the same seed. Six development contexts, 8 seeds each, plus three text prompts. Token identity has no null reference: two runs of the unmodified model were not compared.

false answers; the layer-31 version inserts one "Wait" in every output, after which the model sometimes recognises the wrong month on Budget and rationalises it. Even the input such a circuit aims for does not help: injecting a generic instruction to compare each requested detail with the visible evidence into the reasoning, with the original weights, late or early, gives 5/5 and 5/5 false answers. A narrow MLP edit adapted from the weight-space programming recipe (64 columns in each of layers 17 and 20, 327,680 coefficients targeted) gives 8/8, 8/8, 8/8, 7/8 premature answers at $\alpha = 0.5$, 1, 1.5 and 6, and 0 useful actions in any arm; a random direction at $\alpha = 6$ gives 7/8, and true completions stay at 9–10/12 against 10/12. The last hypotheses closed the same way. Promoting the end-of-thinking token so that the model reasons less leaves `answer` rates unchanged at the dose that keeps outputs valid (false completions 13/20 against 13/20, true completions 18/20 against 18/20, on a 28-context panel), and breaks the JSON at higher doses (2/20 and 2/4 invalid outputs before the arms were stopped). In 3 further edits (penalising completion words, amplifying three task-reading heads, interpolating late MLPs toward Qwen), every dose gives 0/8 useful actions. An adaptation of TACT [29], which steers overthinking and overacting at the end of reasoning, could not be built as specified: after a blind audit of the labels, 13 overthinking and 7 overacting states remain, and the overthinking axis (AUROC 0.983) barely beats position and reasoning length (0.949). Pushing the MLP edit all the way breaks the model rather than the decision: with the MLPs set exactly to Qwen's, 34/48 outputs are invalid (its only `answer` calls are on the infeasible context), and one pilot output repeats a single token over 91.6% of its 3,072 tokens.

These screens share the limits of §6.3: 16 samples per stratum come from two contexts, the circuit arms stopped after 5 outputs once the gate could no longer be met and never measured protected classes, and the MLP edit is an adaptation of the published recipe, not a numerical reproduction.

Weight edit	tool calls				blind audit, fs+		identical to original	
	fs+ `answer`	fs− `answer`	healthy valid	invalid	false claim	useful	tokens	reasoning
Original (re-served)	16/16	15/16	16/16	0/48	16/16	0/16	–	–
DeltaNet gates → Qwen (H2, 48 matrices)	16/16	15/16	16/16	0/48	16/16	0/16	2/51	21/51
MLPs halfway to Qwen (H3, 96 matrices)	16/16	15/16	16/16	0/48	16/16	0/16	0/51	1/51
`</think>` input row → newline (H5, $\lambda = 0.5$)	16/16	15/16	16/16	0/48	16/16	0/16	4/51	39/51
Full-attention temperature $\times 0.5$ (H6, 8 vectors)†	13/16	16/16	16/16	0/48	13/16	0/16	–	–
DeltaNet gates, $\lambda = 2$ (H2, 48 matrices)	16/16	15/16	16/16	0/48	16/16	0/16	–	–
Language layers 20% toward Qwen (H9, 401 tensors)	16/16	16/16	16/16	0/48	16/16	0/16	–	–
RoPE share of the query gain $\times 0.5$ (H10, 8 vectors)	16/16	15/16	16/16	0/48	16/16	0/16	–	–
RoPE share of the query gain $= 0$ (H10)†	8/16	13/16	14/16	3/48	8/16	0/16	–	–
MLPs = Qwen (H3, $\lambda = 1$)	3/16	3/16	8/16	34/48	–	–	–	–

Table 7: Mechanism-level weight edits change the tokens, not the outcome. vLLM 0.23 on separate hardware, same 6 contexts and 8 seeds per arm; the original is re-served on the same backend. The blind audit reads each false-completion output and counts claims of success and useful GUI actions, whatever tool is called. "Identical to original" counts the 48 GUI and 3 text outputs equal to the paired original, in full or up to the end of reasoning. One of the two FS+ contexts is an infeasible task, where `answer` explaining the impossibility is the correct action (Appendix Table 5). †Moving the tool is not a correction: every non-`answer` call falls on the infeasible context and is an unhelpful action justified by an incorrect prediction that it will restore the old interface (H6: 3 theme clicks; H10, $s = 0$: 7 theme clicks and 1 wait), none admitting the impossibility; none is useful. H10 at $s = 0$ is also rejected for truncated outputs on healthy steps, a loss of at least 12.5 points against a 10-point limit. Development pilot.

C Compiling an intervention into the weights

No steering candidate passed the gate, so none was compiled; the weight edits of §6.5 were built directly by their own constructions. We state the compilation step because it constrains which steering interventions are worth testing. Let the residual stream at layer ℓ be $x_\ell = e + \sum_{j<\ell}(a_j + m_j)$, where $a_j = W_j^O z_j^a$ and $m_j = W_j^{\text{down}} z_j^m$ are the attention and MLP writes and e the token or image embedding, and let $P = I - uu^\top$.

Proposition 1 (Directional ablation equals writer orthogonalisation; cf. [2])**.** *In a pre-norm decoder whose sublayers read the residual stream only through their normalisation and whose writers have no bias, replacing every writer $W \in \{W_j^O, W_j^{down}, W^V\}$ by PW, the embedding by $W_E P$ and the vision-merger bias by Pb^V gives a forward pass identical to the original model with $x \leftarrow Px$ applied after every write, at every layer and position.*

Proof sketch. By induction over writes: if x has no u-component, every normalised sublayer input is unchanged, so every z is unchanged, and each new write $PWz = P(Wz)$ keeps the sum orthogonal to u. □

Four consequences follow. A hook at block outputs is not equivalent, because the MLP then reads the un-ablated attention output. $W \leftarrow (I - \alpha uu^\top)W$ with $\alpha \neq 1$ scales each write's component by $1 - \alpha$, whereas a residual hook compounds over layers. Tied embeddings (tied in both models) mean that projecting W_E also projects the output head unless it is untied. And an addition $h \leftarrow h + \beta u$, like every intervention in Table 2 except projections, can be compiled only as a bias, which the bias-free projections of Holo-3.1 do not have. The circuit arm is the exception: scaling heads and neurons is exactly a column scaling of W^O and W^{down}, 1,991,680 weights or 0.0439% of the model, and it has been tested in generation with no effect. Any compiled patch must reproduce the steered behaviour within 5 points per stratum before serving. The serving path is ready: a patched checkpoint is loaded by vLLM in about seven minutes, a null patch changes 0/724 tensors and reproduces the original outputs. Since no candidate passed, no compiled edit or equivalence check was run.

D OSWorld diagnostic by domain

Table 8 breaks down the categories of §3 by OSWorld domain.

Domain	Tasks	Success	FS	Step lim.	Loop	Other + unsc.
multi-apps	31	13/30	6	9	0	2
chrome	15	9/15	4	1	1	0
lo-calc	15	8/15	5	1	1	0
lo-impress	15	10/15	2	1	1	0
gimp	9	4/9	4	1	0	0
lo-writer	8	5/8	1	1	1	0
os	8	4/8	2	2	0	0
vs-code	8	6/8	1	0	1	0
vlc	6	2/6	4	0	0	0
thunderbird	5	5/5	0	0	0	0
Total	120	66/119	29	16	5	2

Table 8: Diagnostic split by domain (Holo-3.1-4B, pass 1). FS: false success; Other + unsc.: unclassified failures plus unscored tasks; lo: LibreOffice.

E Value-axis construction

The value direction of `astra-value` contrasts success with failure steps of the same relative stage, averaged within task, on 68 construction tasks; the commitment direction contrasts `answer` with continuation steps. Neither uses the FS+/FS− labels. Evaluated on FS+ versus FS− under task-grouped cross-validation, the value direction scores below 0.5 at layers 18, 24 and 28, so it is higher on true completions, while step index and trajectory length reach about 0.85. Its separation is thus weaker than position, the same confound as Trap 1.

F Holo-3.1-0.8B on MiniWoB++: details

The tables below give the 0.8B study of §8 in full: the diagnostic (Table 9), the offline screening of weight edits (Tables 11 and 14), the steering arms (Table 12) and the per-task results (Table 13).

Primary category	Rollouts	% of failures
False success	37	36.3
Loop	25	24.5
Grounding miss	21	20.6
Abandon / budget	10	9.8
Wrong final action	9	8.8
Total failures	102	100.0

Table 9: Primary failure category, 0.8B (102 failed rollouts).

Layer	Contexts	min cosine	max rel. error
0	40	0.99995	0.0104
10	40	0.99951	0.0314
23	40	0.99694	0.0787

Table 10: Replay fidelity on MiniWoB++: replayed vs. recorded residuals.

Split	Steps	n	Decrease	95% CI (task bootstrap)	Random dir.
Dev (seeds 0–5)	loop+	72	3.89	[1.32, 6.41]	–
	healthy	344	9.97	[6.62, 13.69]	–
	loop−	287	9.16	[6.51, 11.63]	–
	success	119	10.86	[3.99, 20.58]	–
Reserved (seeds 6–7)	loop+	23	2.31	[-0.67, 7.62]	1.30
	healthy	119	8.92	[5.07, 14.03]	4.19
	loop−	90	10.33	[6.73, 14.81]	4.20
	success	54	9.42	[2.68, 19.39]	5.33

Table 11: Frozen 0.8B weight edit vs. original: mean decrease in log-probability per token of the recorded action (mnat/token), task-level bootstrap CIs. The random-direction column uses the same layers, writers, α and top-k.

Arm	REPEAT % [95% CI]	HEALTH % [95% CI]	Gens	Status
Original (no hook)	95.8 [88, 100]	75.0 [58, 90]	96	complete
Original, re-run	95.8 [88, 100]	70.8 [52, 88]	96	complete
Hook, $\alpha = 0$	95.8 [88, 100]	72.9 [54, 90]	96	complete
Random dir., all layers	95.8 [88, 100]	72.9 [54, 90]	96	complete
u_{10} at L10	93.8 [85, 100]	66.7 [48, 85]	96	complete
u_{10} at L9–13	89.6 [79, 100]	60.4 [40, 79]	96	complete
u_{10}, all layers	93.8 [83, 100]	64.6 [42, 85]	96	complete
u_{10}, all layers, $\alpha = 2$	93.8 [83, 100]	56.2 [35, 75]	96	complete
per-layer u_ℓ, all layers	81.2 [65, 94]	58.3 [40, 77]	96	complete
matched-pair dir.	n/m	$\leq$ 66.7	36	rejected early
matched-pair PCA	n/m	$\leq$ 62.5	36	rejected early
MLP-site dir.	n/m	$\leq$ 66.7	36	rejected early
MLP-site PCA	n/m	$\leq$ 62.5	40	rejected early
mean of layer dirs.	n/m	$\leq$ 64.6	40	rejected early
strict pairs	n/m	$\leq$ 62.5	36	rejected early
strict pairs PCA	n/m	$\leq$ 64.6	40	rejected early
strict MLP	n/m	$\leq$ 68.8	36	rejected early
strict MLP PCA	n/m	$\leq$ 66.7	36	rejected early
u_{10} amplification $\alpha = -1$	n/m	n/m	4	aborted (invalid)
constant addition $-0.25\, u_{10}$	n/m	n/m	4	aborted (invalid)

Table 12: 0.8B free-generation steering arms (12 loop + 12 healthy contexts, $k = 4$). "Rejected early" arms show an upper bound on HEALTH that already fails the threshold.

Task	Success	Task	Success
`click-option`	7/7	`click-menu`	2/7
`click-tab`	8/8	`use-autocomplete`	2/7
`email-inbox-delete`	7/7	`click-tab-2`	1/7
`click-checkboxes`	7/8	`form-sequence`	1/7
`click-widget`	7/8	`login-user`	1/7
`navigate-tree`	6/7	`login-user-popup`	1/7
`click-dialog-2`	6/8	`multi-layouts`	1/7
`focus-text-2`	3/7	`social-media`	1/7
`enter-text-2`	3/8	`click-collapsible`	0/7
`click-checkboxes-soft`	2/6	`enter-date`	0/7
`choose-list`	2/7	`enter-password`	0/7
`click-button-sequence`	2/7	`search-engine`	0/7

Table 13: 0.8B success per MiniWoB++ task.

Candidate	Layers	α	top-k	Writers	Weights	$D_{\text{loop+}}$	D_{healthy}	$D_{\text{loop}-}$	D_{succ}	Pass
r3_distributed_a16	9–23	16.0	64	mlp+attn	1,966,080	262.66	286.96	259.44	342.09	no
r3_early_a16	9–10	16.0	256	mlp+attn	1,048,576	206.20	228.26	197.14	284.70	no
r3_L10_a16	10	16.0	64	mlp+attn	131,072	27.34	34.61	31.67	43.46	no
r3_distributed_a4	9–23	4.0	64	mlp+attn	1,966,080	13.96	6.69	8.61	3.29	no
a2_r3_a3	9–23	3.0	64	mlp+attn	1,966,080	9.31	-1.07	2.23	-6.32	yes
r3_early_a4	9–10	4.0	256	mlp+attn	1,048,576	8.11	14.12	8.68	17.66	no
a2_r3_a25	9–23	2.5	64	mlp+attn	1,966,080	7.08	-2.74	0.18	-7.62	yes
r3_mit_layers_a16	15,19	16.0	64	mlp	131,072	6.53	1.14	-1.04	2.85	no
a2_r2_distributed_a2	9–23	2.0	64	mlp+attn	1,966,080	4.72	-3.71	-1.26	-8.14	yes
r2_distributed_None	9–23	1.0	all	mlp+attn	86,507,520	4.47	1.05	1.89	0.25	no
a2_r3_mlp	9–23	2.0	64	mlp	983,040	3.40	-2.03	0.55	-5.25	yes
r3_L10_a4	10	4.0	64	mlp+attn	131,072	3.28	1.43	0.66	1.70	no
r2_early_None	9–10	1.0	all	mlp+attn	11,534,336	2.77	0.13	0.32	-1.45	yes
a2_r3_k32	9–23	2.0	32	mlp+attn	983,040	2.49	-3.59	-2.42	-6.75	yes
r1_L10_mlp_attn_None	10	1.0	all	mlp+attn	5,767,168	2.11	-0.13	0.73	-1.22	no
a2_r3_attn	9–23	2.0	64	attn	983,040	2.07	-2.85	-2.16	-4.49	yes
r2_distributed_64	9–23	1.0	64	mlp+attn	1,966,080	1.99	-5.19	-3.24	-8.60	yes
r2_late_attn_None	10,13,15,17,18,19,20	1.0	all	attn	14,680,064	1.89	-3.80	-2.32	-6.17	yes
a2_r3_k16	9–23	2.0	16	mlp+attn	491,520	1.88	-3.50	-2.93	-5.92	yes
r1_L10_attn_None	10	1.0	all	attn	2,097,152	1.77	-1.08	-0.38	-2.30	yes
r3_mit_layers_a4	15,19	4.0	64	mlp	131,072	1.38	-1.64	-1.08	-1.82	yes
a2_r2_distributed128	9–23	0.7	128	mlp+attn	3,932,160	1.38	-3.61	-3.23	-5.62	yes
r2_late_attn_64	10,13,15,17,18,19,20	1.0	64	attn	458,752	1.37	-3.25	-2.34	-5.11	yes
a2_r1_matched_mlp_attn	10	0.5	64	attn	65,536	1.13	-0.30	-0.52	-0.53	yes
a2_r1_matched_resid_attn	10	0.5	64	attn	65,536	1.02	-0.00	0.33	-0.45	yes
a2_r2_matched_early	9–10	0.7	128	attn	262,144	0.84	-0.12	-0.89	-0.23	yes
r1_L10_mlp_attn_64	10	1.0	64	mlp+attn	131,072	0.84	-0.43	0.04	-1.27	yes
r1_L10_attn_64	10	1.0	64	attn	65,536	0.83	-0.78	-0.67	-1.57	yes
a2_r2_matched_late	10,13,15,17,18,19,20	0.7	64	attn	458,752	0.76	-0.01	0.13	-0.45	yes
a2_r1_mlp_attn	10	0.5	64	attn	65,536	0.68	0.99	0.71	1.08	no
a2_r2_matched_attn16	10	0.3	16	attn	16,384	0.63	-0.44	-0.29	-1.30	yes
r2_mit_layers_None	15,19	1.0	all	mlp	7,340,032	0.63	-0.14	0.53	-0.11	no
r1_L10_mlp_64	10	1.0	64	mlp	65,536	0.53	-0.65	-0.53	-1.14	yes
r2_mit_layers_64	15,19	1.0	64	mlp	131,072	0.53	-0.30	0.05	-0.54	yes
a2_r1_matched_resid_mlp	10	0.5	64	mlp	65,536	0.52	-0.41	0.21	-1.35	no
a2_r2_matched_attn128	10	0.7	128	attn	131,072	0.52	0.03	-0.49	-0.19	yes
a2_r1_mlp_mlp	10	0.5	64	mlp	65,536	0.43	-0.80	-0.64	-1.46	yes
r1_L10_mlp_None	10	1.0	all	mlp	3,670,016	0.34	-0.21	0.14	-0.71	no
r2_early_64	9–10	1.0	64	mlp+attn	262,144	0.30	-1.33	-0.60	-2.79	yes
a2_r1_matched_mlp_mlp	10	0.5	64	mlp	65,536	0.18	-0.72	-0.03	-1.32	yes

Table 14: All offline weight-edit candidates on the 0.8B (40 fixed development contexts). D: decrease in mean log-probability per token of the recorded action (mnat/token). "Pass": $D_{\text{loop+}} > 0$ and $D_{\text{loop+}} \geq 3\max(0, D_{\text{healthy}}, D_{\text{loop}-}, D_{\text{succ}})$.